



MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
(A constituent unit of MAHE, Manipal)

CSE_1081
PROBLEM SOLVING USING
COMPUTERS - LAB MANUAL

FIRST YEAR
COMMON TO ALL BRANCHES
(2022 CURRICULUM)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
MANIPAL INSTITUTE OF TECHNOLOGY
MAHE, BENGALURU
KARNATAKA - 560064

CONTENTS

LAB NO.	TITLE	PAGE NO.	REMARKS
	COURSE OBJECTIVES AND OUTCOMES	i	
	EVALUATION PLAN	i	
	INSTRUCTIONS TO THE STUDENTS	ii-iii	
	SAMPLE LAB OBSERVATION NOTE PREPARATION	iv-v	
1	FLOWCHARTS USING RAPTOR	1	
2	SIMPLE C PROGRAMS	3	
3	KNOWING DATA TYPES	7	
4	PRACTISE ON FORMULA CALCULATIONS	8	
5	BRANCHING CONTROL STRUCTURES	9	
6	LOOPING CONTROL STRUCTURES-FOR LOOPS	13	
7	LOOPING CONTROL STRUCTURES- WHILE & DO LOOPS	15	
8	1D ARRAYS & 2D ARRAYS	18	
9	STRINGS	23	
10	MODULAR PROGRAMMING - FUNCTIONS	25	
11	MODULAR PROGRAMMING- STRUCTURES	28	
12	MODULAR PROGRAMMING- POINTERS	32	
13	LAB EXAM		
	REFERENCES	34	

	C LANGUAGE QUICK REFERENCE	35	
	EXERCISE SOLUTIONS	41	

Course Objectives

- To develop the programming skills using basics of C language
- To write algorithms and draw flowchart for various problems
- To write, compile and debug programs in C language

Course Outcomes

At the end of this course, students will be able to

- Represent and manipulate data to store it in a computer using C syntax.
- Develop, execute, and document C programs.

Evaluation plan

- Internal Assessment Marks: 60%
 - ✓ Students must work out the C programs in code blocks only.
 - ✓ Students shall submit the lab code along with the results obtained under every program **as per the schedule**. Eg: At the end of lab 2, both lab 1 & lab 2 programs should be submitted.
- End semester assessment of 2-hour duration: 40%
 - ✓ A complete C program using functions will be asked. The given program must be worked out in code blocks and compiled. The **C program code** should be submitted. The name of file should be like **regno.c Eg: 20100345.c**

Note:

1. Students shall do all the programming exercises in CODE BLOCKS only. The URL for downloading and procedure is **given below**.
2. An additional lab exercise on Structures / Pointers (self-study) is kept at the end for student's reference.

Procedure for CODE BLOCKS downloading and installing.

Code:Blocks is a free C/C++ and Fortran IDE built to meet the most demanding needs of its users. It is designed to be very extensible and fully configurable.

**URL: <https://www.codeblocks.org/downloads/binaries/>
<https://www.codeblocks.org/downloads/>**

In this URL you may be able to download for the following operating systems.

- **Windows XP / Vista / 7 / 8.x / 10**
- **Linux 32 and 64-bit**
- **Mac OS X**

Please look into the below YouTube link which details how to download and install code blocks for windows 10.

YouTube: <https://www.youtube.com/watch?v=GWJgsmitR2I>

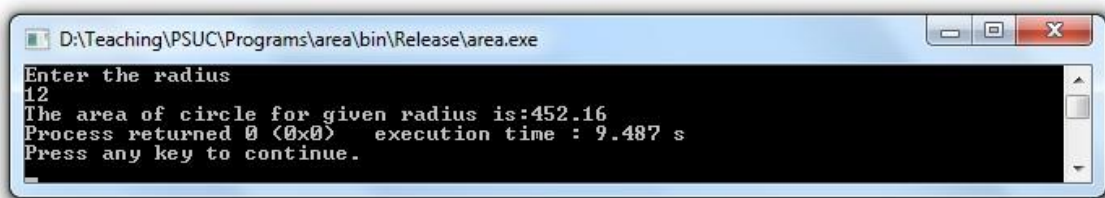
Sample Lab Observation Note Preparation

Title: SIMPLE C PROGRAMS

1. Program to find area of the circle. (Hint: $\text{Area} = 3.14 * r * r$)

```
//program to find area of circle
#include<stdio.h>
int main()
{
    int radius;
    float area;
    printf("Enter the radius\n");
    scanf("%d", &radius);
    area=3.14*radius*radius;
    printf("My name is abcd"); →Name should be printed in every prog.
    printf("The area of circle for given radius is: %f", area);
    return 0;
}
```

Sample input and output: Screen shot of result – must be included.



The screenshot shows a Windows command prompt window with the title bar "D:\Teaching\PSUC\Programs\area\bin\Release\area.exe". The window contains the following text:

```
Enter the radius
12
The area of circle for given radius is:452.16
Process returned 0 (0x0)   execution time : 9.487 s
Press any key to continue.
```

LAB NO.: 1

FLOWCHART BASICS

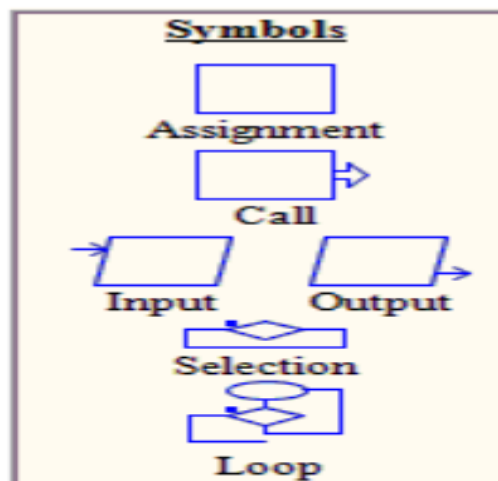
Objectives

In this lab, student will be able to:

1. Build flowcharts using Raptor.
2. Compile and execute flowcharts.
3. Verify Input and Output.

Raptor: Basic Symbols used in raptor window

The six symbols used in Raptor are displayed in the Symbol Window in the upper left corner of the main window:



Assignment Symbol

The assignment symbol is used to change the value of a variable. The right hand side of the assignment is evaluated, and the resulting value is placed in the variable on the left hand side. For example, consider the case where the value of x is currently 5, and the assignment " $x \leftarrow x + 1$ " is executed. First " $x+1$ " is evaluated, yielding the result 6. Then the value of x is changed to be 6. Note that assignment is very different from

mathematical equality. The statement should be read "Set x to x+1" instead of "x equals x+1". The assignment symbol can also be used to assign a string expression to a string variable.

Call Symbol

prompted with a dialog to enter a value that can be interpreted as either a number or string, depending on what the user types (see String vs. Numeric Input). The user can also override the source for input by specifying a text file to be used in place of the keyboard.

Output Symbol

The output symbol is used to either write a number or text to the Master Console window. The user can also override the destination for output by specifying a text file to be used instead of the Master Console.

Selection Symbol

The selection structure is used for decision making. The programmer enters in the diamond an expression that evaluates to Yes (True) or No (False). Such expressions are formally referred to as Boolean expressions. Based on the result of the expression in the diamond, control of the program will branch either left (Yes, or True) or right (No, or False).

Loop Control Symbol

The loop structure is used to repeat a sequence of symbols until a certain condition is met. When execution reaches the bottom of the loop, it starts over again at the top. The loop is exited when the diamond symbol is executed and the Boolean expression in the diamond evaluates to Yes (True).

Lab Exercises

1. To add two integers a and b read through the keyboard. Display the result using third variable sum
2. Calculate area of the circle.
3. To find largest of two numbers.

LAB NO.: 2

Objectives

In this lab, student will be able to:

1. Write C programs.
2. Compile and execute C programs.
3. Debug and trace the programs.

Code: Blocks Integrated Development Environment

Code: Blocks has a C editor and compiler. It allows us to create and test our programs. Code: Blocks creates Workspace to keep track of the project that is being used. A project is a collection of one or more source files. Source files are the files that contain the source code for the problem.

Let's look at C program implementation in steps by writing, storing, compiling and executing a sample program

- Create a directory with section followed by roll number (to be unique); e.g. A21.
- As per the instructions given by the lab teacher, create *InchToCm.c* program.
 - Open a new notepad file and type the given program
- Save the file with name and extension as "*InchToCm.c*" into the respective directory created.

Sample Program (*InchToCm.c*):

```
// InchToCm.c
#include <stdio.h>
int main()
{
    float centimeters, inches;

    printf( "This program converts inches to centimeters"\n);
    printf( "Enter a number");
    scanf("%f", &inches);
```

```

        centimeters = inches * 2.54;
        printf("%f inches is equivalent to %f centimeters\n", inches, centimeters);
        return 0;
    } // end main

```

- Run the program as per the instructions given by the lab teacher.
 - Compile the saved program and run it either by using keyboard short cuts or through the menu.

PROGRAM STRUCTURE AND PARTS

Comments

The first line of the file is:

```
// InchToCm.c
```

This line is a comment. Let's add a comment above the name of the program that contains the student's name.

- Edit the file. Add the student's name on the top line so that the first two lines of the file now look like:

```
// student's name
```

```
// InchToCm.c
```

Comments tell people reading the program something about the program. The compiler ignores these lines.

Preprocessor Directives

After the initial comments, the student should be able to see the following lines:

```
#include <stdio.h>
```

This is called a preprocessor directive. It tells the compiler to do something. Preprocessor directives always start with a # sign. The preprocessor directive includes the information in the file *stdio.h*. as part of the program. Most of the programs will almost always have at least one include file. These header files are stored in a library that shall be learnt more in the subsequent labs.

The function main ()

The next non-blank line *int main ()* gives the name of a function. There must be exactly one function named *main* in each C program, and *main* is the function where program execution starts when the program begins running. The *int* before *main ()* indicates the function is returning an integer value and also to indicate empty argument list to the function. Essentially functions are units of C code that do a particular task. Large programs will have many functions just as large organizations have many functions. Small programs, like smaller organizations, have fewer functions. The parentheses following the words *int main* contains a list of arguments to the function. In the case of this function, there are no *arguments*. Arguments to functions tell the function what objects to use in performing its task. The curly braces ({} on the next line and on the last line { }) of the program determine the beginning and ending of the function.

Variable Declarations

The line after the opening curly brace, *float centimeters, inches;* is called a variable declaration. This line tells the compiler to reserve two places in memory with adequate size for a real number (the *float* keyword indicates the variable as a real number). The memory locations will have the names *inches* and *centimeters* associated with them. The programs often have many different variables of many different types.

EXECUTABLE STATEMENTS

Output and Input

The statements following the variable declaration up to the closing curly brace are executable statements. The executable statements are statements that will be executed when the program run. *printf ()* statement tells the compiler to generate instructions that will display information on the screen when the program run, and *scanf ()* statement reads information from the keyboard when the program run.

Format Specifiers	Description	Example
%d	%d is used to print the value of integer variable. We can also use %i to print integer value. %d and %i have same meaning	int v; scanf("%d",&v); printf("value is %d", v);
%f	%f is used to print the value of floating point variable in a decimal form.	float v; scanf("%f",&v); printf("value is %f",v);
%c	%c is used to print the value of character variable.	char ch; scanf("%c",&v); printf("value is %c",ch)
%s	%s is used to print the string	char name[20]; scanf("%s",v); printf("value is %s",name)

Assignment Statements

The statement *centimeters = inches * 2.54;* is an assignment statement. It calculates what is on the right hand side of the equation (in this case *inches * 2.54*) and stores it in the memory location that has the name specified on the left hand side of the equation (in this case, *centimeters*). So *centimeters = inches * 2.54* takes whatever was read into the memory location *inches*, multiplies it by 2.54, and stores the result in *centimeters*. The next statement outputs the result of the calculation.

Return Statement

The last statement of this program, *return 0;* returns the program control back to the operating system. The value 0 indicates that the program ended normally. The last line of every main function written should be *return 0;*

Syntax

Syntax is the way that a language must be phrased in order for it to be understandable.

The general form of a C program is given below:

```
// program name
// other comments like what program does and student's name
# include <appropriate files>
int main()
{
    Variable declarations;
    Executable statements;
} // end main
```

Lab Exercises

1. Write a C program to add two integers a and b read through the keyboard. Display the result using third variable sum
2. Calculate area of the circle.
3. WAP to find largest of two numbers.

LAB NO.: 3

Objectives

In this lab, student will be able to:

1. Write C programs to know the data types.
2. Compile and execute C programs.
3. Debug and trace the programs.

Lab Exercises

1. WAP to print ASCII values.
2. WAP to display size of the datatypes int, char, float, double, long int and long double using size of () operator.
3. Convert temperature Fahrenheit to Celsius

LAB NO.: 4

Objectives

In this lab, student will be able to:

1. Write C programs to Practice on formula calculation.
2. Compile and execute C programs.
3. Debug and trace the programs.

Lab Exercises

1. Write a C program to evaluate the following expression for the values $a = 30$, $b=10$, $c=5$, $d=15$
(i) $(a + b) * c / d$ (ii) $((a + b) * c) / d$
(iii) $a + (b * c) / d$ (iv) $(a + b) * (c / d)$
2. Given number is even or odd.
3. Swap two numbers w/o using third variable

Additional Exercises

1. Write a c program that ask interactively the users name and age and responds with : Hello name, next year you will be next_age (where next _age is age +1).
2. Write a program to calculate the distance between two points.
3. Write a program to perform addition, subtraction, division, multiplication and modulo division on two integers
4. Write a program to print the digit at ones place of a number.
5. Write a program to calculate the bill amount for an item given its quantity sold, value, discount and tax.
6. Write a program to calculate salary of an employee given his basic pay(to be entered by user), HRA=10%- of the basic pay, TA=5% of basic pay. Define HRA and TA as constants and use them to calculate the salary of the employee.
7. Write a program to prepare a grocery bill. For that enter the name of the items purchased, quantity in which it is purchased, and its price per unit. Then display the bill in the following format:

***** BILL *****

Item	Quantity	Price	Amount
------	----------	-------	--------

Total	Amount to be paid
-------	-------------------

LAB NO.: 5

BRANCHING CONTROL STRUCTURES

Objectives:

In this lab, student will be able to do C programs using

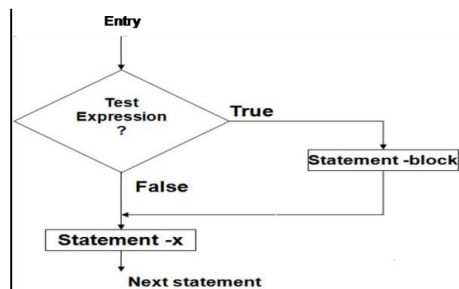
1. **simple if** statement
2. **if-else** statement
3. **switch-case** statement

Introduction:

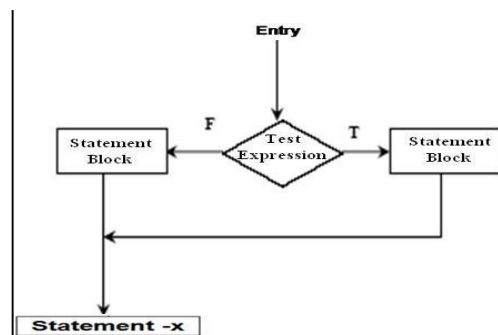
- A control structure refers to the way in which the programmer specifies the order of execution of the instructions

C decision making and branching statements flow control actions:

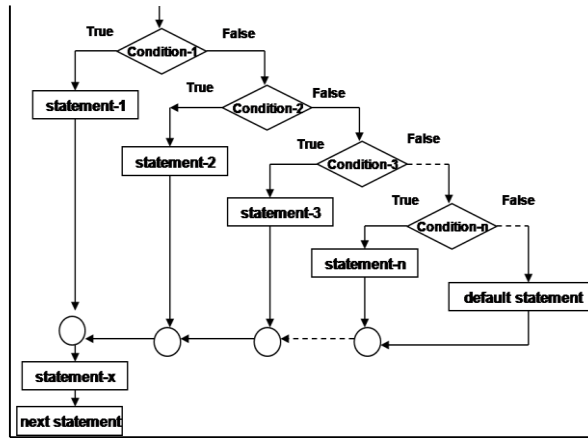
Simple if statement:



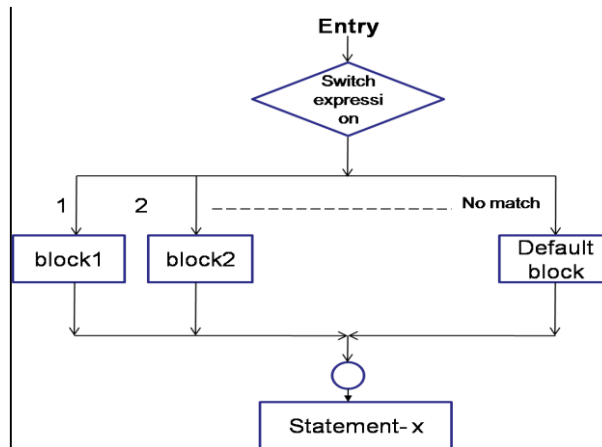
If - else statement:



Else - if ladder:



Switch statement:



Solved Exercise

C program to compute all the roots of a quadratic equation

```
#include<stdio.h>
```

```
#include<math.h>
```

```
int main() {
```

```
int a,b,c;
```

```
float root1, root2, re, im, disc;
```

```
scanf("%d,%d,%d",&a,&b,&c);
```

```

disc=b*b-4*a*c;
if (disc<0) // first if condition
{
printf("imaginary roots\n");
re= - b / (2*a);
im = pow(fabs(disc),0.5)/(2*a);
printf("%f +i %f",re,im);
printf("%f -i %f",re,im);
//printf("%f, re", "+ i",im);
//cout<<re<<"-i"<<im;
}
else if (disc==0){ //2nd else-if condition
printf("real & equal roots");
re=-b / (2*a);
printf("Roots are %f",re);
}
else{ /*disc > 0- otherwise part with else*/
printf("real & distinct roots");
printf("Roots are");
root1=(-b + sqrt(disc))/(2*a);
root2=(-b - sqrt(disc))/(2*a);
printf("%f and %f",root1,root2);
}
return 0;
}

```

Lab Exercises

With the help of various branching control constructs like **if**, **if-else** and **switch** case statements,

Write C programs to do the following:

1. Compute all the roots of a quadratic equation using **switch case** statement.[Hint: $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$]
2. Find the smallest among three numbers using conditional operator.

Additional Exercises

1. Write a c program to determine whether a person is eligible to vote.
2. Write a program to find whether a given number is odd or even.
3. Write a program to enter a character and then determine whether it is a vowel or not.
4. Write a program to find whether a given year is a leap or not.
5. Write a program to enter the marks of a student in four subjects. Then calculate the total, aggregate and display the grades obtained by the student.
6. Write a program to enter a number from 1-7 and display the corresponding day of the week using switch statement.
7. Write a program which can check a number given by the user is zero, positive or negative.

LAB NO.: 6

LOOPING CONTROL STRUCTURES- FOR LOOPS

Objectives:

In this lab, student will be able to:

- Write and execute C programs using 'for' statement
- To learn to use break and continue statements in for loop statements.

Introduction:

- For loop statements are used to repeat certain statements for a specified number of times.
- The statements are executed as long as the execution condition is true
- These types of control structures are also called as loop control structures

For loop:

```
for (initialization; test condition; increment/decrement)
{
    body of the loop
}
```

With the help of *for loop* statements,

1. Generate the multiplication table for '*n*' numbers up to '*k*' terms (using nested for loops).

```
[ Hint: 1  2  3  4  5  ....  k
        2  4  6  8 10  ....2*k
        .....
        n..... n*k ]
```

2. Generate Floyd's triangle using natural numbers for a given limit N. (using for loops)
[Hint: Floyd's triangle is a right angled-triangle using the natural numbers]

Ex: Input: N = 4

Output:

1
2 3
4 5 6
7 8 9 10

3. Evaluate the sine series, $\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$ to n terms.

4. Check whether a given number is perfect or not.

[Hint: Sum of all positive divisors of a given number excluding the given number is

equal to the number] Ex: $28 = 1 + 2 + 4 + 7 + 14 = 28$ is a perfect number

5. Find out the generic root of any number.

[Hint: Generic root is the sum of digits of a number until a single digit is obtained.]

Ex: Generic root of 456 is $4 + 5 + 6 = 15 = 1 + 5 = 6$

6. Write a program to demonstrate use of break and continue statements in **for** loop

Lab Exercises

1. WAP to reverse the number and also check the is it palindrome.
2. Check whether the number is strong number or not.
3. WAP check the number is Armstrong or not.
4. Use break and continue keyword.

Additional Exercises

1. Write a c program to print the sum of digits of a number.
2. Write a program to find factorial of a given number.
3. Write a program to calculate the average of numbers entered by the user.
4. Write a program to enter a decimal number, calculate and display the binary equivalent.
5. Write a program to generate calendar of a month given the start day of the week and the number of days in that month.

LAB NO.: 7

LOOPING CONTROL STRUCTURES-WHILE & DO LOOPS

Objectives:

In this lab, student will be able to:

1. Write and execute C programs using 'while' statement
2. Write and execute C programs using 'do-while' statement
3. To learn to use break and continue statements in while and do while loop statements.

Introduction:

- Iterative (repetitive) control structures are used to repeat certain statements for a specified number of times.
- The statements are executed as long as the condition is true
- These types of control structures are also called as loop control structures
- Three kinds of loop control structures are:
 - while
 - do-while
 - for

C looping control structures:

While loop:

```
while (test condition)
{
    body of the loop
}
```

Do-while loop:

```
do
{
    body of the loop
}
while (test condition);
```

Solved Exercise

[Understand the working of looping with this illustrative example for finding sum of natural numbers up to 100 using **while** and **do-while** statements]

Using **do-while**

```
#include<stdio.h>
int main()
{
    int n;
    int sum;
    sum=0; //initialize sum
    n=1;
    do
    {
        sum = sum + counter;
        counter = counter +1;
    } while (counter < 100);
    printf(“%d”,sum);
    return 0;}
```

Using **while**

```
#include<stdio.h>
int main( )
{
    int n;
    int sum;
    sum=0; //initialize sum
    n=1;
    while (n<100)
    {
        sum = sum + n;
        n = n +1;
    }
```

```
printf("%d",sum);
return 0; }
```

Write C programs to do the following with the help of iterative (looping) control structures such as **while and do-while** statements,

1. Reverse a given number and check if it is a palindrome or not. (use while loop).[Ex: 1234, reverse= $4 \times 10^3 + 3 \times 10^2 + 2 \times 10^1 + 1 \times 10^0 = 4321$]
2. Generate prime numbers between 2 given limits.(use while loop)
3. Check if the sum of the cubes of all digits of an inputted number equals the number itself (Armstrong Number). (use while loop)
4. Write a program using do-while loop to read the numbers until -1 is encountered. Also count the number of prime numbers and composite numbers entered by user. [Hint: 1 is neither prime nor composite]
5. Check whether the given number is strong or not.
6. [Hint: Positive number whose sum of the factorial of its digits is equal to the number itself] Ex: $145 = 1! + 4! + 5! = 1 + 24 + 120 = 145$ is a strong number.
7. Write a program to demonstrate use of break and continue statements in while and do-while loops.

Lab Exercises

1. Given number is a perfect number.
2. N tables up to k terms using two for loops.
3. Floyd's triangle for the given no. of rows.
4. Find out prime no. using while loop

LAB NO.: 8

1D ARRAYS

Objectives:

In this lab, student will be able to:

- Write and execute programs on 1Dimensional arrays

Introduction to 1D Arrays

1 Dimensional Array

Definition:

- An array is a group of related data items that share a common name.
- The array elements are placed in a contiguous memory location.
- A particular value in an array is indicated by writing an integer number called index number or subscript in square brackets after the array name. The least value that an index can take in array is 0.

Array Declaration:

data-type name [size];

- ✓ where data-type is a valid data type (like int, float,char...)
- ✓ name is a valid identifier
- ✓ size specifies how many elements the array has to contain
- ✓ size field is always enclosed in square brackets [] and takes static values.

Total size of 1D array:

The Total memory that can be allocated to 1D array is computed as:

Total size =size *(sizeof(data_type));

where, *size* is number of elements in 1-D array

data_type is basic data type.

Sizeof() is an unary operator which returns the size of expression or data type in bytes.

For example, to represent a set of 5 numbers by an array variable ***Arr***, the declaration the variable ***Arr*** is

int Arr[5];

Solved Exercise

Sample Program to read n elements into a 1D array and print it:

```
#include<stdio.h>
int main()
{
    int a[10], i, n;
    printf("enter no of elements");
    scanf("%d",&n);
    printf("enter n values\n");
    for(i=0;i<n;i++) // input 1D array
        scanf("%d",&a[i]);
    printf("\nNumbers entered are:\n");
    for(i=0;i<n;i++) // output 1D array
        printf("%d",a[i]);
    return 0;
}
```

Output:

Enter no. of elements

3

Enter n values

9

11

13

Numbers entered are:

9

11

13

Lab Exercises

With the knowledge of 1D arrays, Write C programs to do the following:

1. Find the largest and smallest element in a 1D array.
2. Print all the prime numbers in a given 1D array.
3. Arrange the given elements in a 1D array in ascending and descending order using bubble sort method. [Hint: use switch case (as case 'a' and case'd') to specify the order].
4. Insert an element into a 1D array by getting an element and the position from the user.
5. Search the position of the number that is entered by the user and delete that number from the array and display the resultant array elements.

Objectives:**2D ARRAYS**

In this lab, student will be able to:

- Write and execute programs on 2D dimensional arrays

Introduction to 2 Dimensional Arrays

- It is an ordered table of homogeneous elements.
- It can be imagined as a two dimensional table made of elements, all of them of a same uniform data type.
- It is generally referred to as matrix, of some rows and some columns. It is also called as a two-subscripted variable.

For example

```
int marks[5][3];
```

```
float matrix[3][3];
```

```
char page[25][80];
```

- The first example tells that marks is a 2-D array of 5 rows and 3 columns.
- The second example tells that matrix is a 2-D array of 3 rows and 3 columns.
- Similarly, the third example tells that page is a 2-D array of 25 rows and 80 columns.

Solved Exercise:

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int i,j,m,n,a[100][100];
```

```
printf("enter dimension of matrix");
```

```
scanf("%d %d",&m,&n);
```

```
printf("enter the elements");
```

```
for(i=0;i<m;i++) // input 2D array using 2 for loops
```

```
{
```

```

for(j=0;j<n;j++)
    scanf("%d",&a[i][j]);
}
for(i=0;i<m;i++) // output 2D array with 2 for loops
{
    for(j=0;j<n;j++)
        printf("%d\t",a[i][j]);
    printf("\n");
}
return 0;
}

```

Lab Exercises

With the knowledge of 2D arrays,

Write C programs to do the following:

1. Find whether a given matrix is symmetric or not. [Hint: $A = A^T$]
2. Find the trace and norm of a given square matrix.
[Hint: Trace= sum of principal diagonal elements
Norm= $\text{SQRT}(\text{sum of squares of the individual elements of an array})$]
3. Perform matrix multiplication.
4. To interchange the primary and secondary diagonal elements in the given Matrix.
5. Interchange any two Rows & Columns in the given Matrix.
6. Search for an element in a given matrix and count the number of its occurrences.

LAB NO.: 9

STRINGS

Objectives:

In this lab, student will be able to:

1. Declare, initialize, read and write a string
2. Write C programs with and without string handling functions to manipulate the given string

Introduction

- A string is an array of characters.
- Any group of characters (except double quote sign) defined between double quotations is a constant string.
- Character strings are often used to build meaningful and readable programs.

The common operations performed on strings are

- Reading and writing strings
- Combining strings together
- Copying one string to another
- Comparing strings to another
- Extracting a portion of a string etc.

Declaration

Syntax: ***char string_name[size];***

- The size determines the number of characters in the string_name.

Solved Exercise

Program to read and display a string

```
#include<stdio.h>
int main()
{
    const int MAX = 80; //max characters in string
    char str[MAX];      //string variable str
```

```
printf("Enter a string: ");
scanf("%s",str);
//put string in str
Printf("You entered: %s\n" str); //display string from str
return 0;}
```

Lab Exercises

With the brief introduction and knowledge on strings,

Write Cprograms without using STRING-HANDLING functions for the following:

1. Count the number of words in a sentence.
2. Input a string and toggle the case of every character in the input string.
Ex: INPUT: aBcDe
 OUTPUT: AbCdE
3. Check whether the given string is a palindrome or not.
4. Arrange 'n' names in alphabetical order (hint: use string handling function-**strcpy**)
5. Delete a word from the given sentence.
Ex: INPUT: I AM STUDYING IN MIT
 TO BE DELETED: STUDYING
 OUTPUT: I AM IN MIT

LAB NO.10

MODULAR PROGRAMMING -FUNCTIONS

Objectives:

In this lab, student will be able to:

1. Understand modularization and its importance
2. Define and invoke a function
3. Analyze the flow of control in a program involving function call
4. Write programs using functions

Introduction

- A **function** is a set of instructions to carry out a particular task.
- Using functions programs can be structured in a **more modular** way.

Function definition and call

```
// FUNCTION DEFINITION
Return type  Function name  Parameter List
void DisplayMessage(void)
{
    cout << "Hello from function DisplayMessage\n";
}

int main()
{
    cout << "Hello from main";
    DisplayMessage(); // FUNCTION CALL
    cout << "Back in function main again.\n";
    return 0;
}
```

Solved Exercise

1. Program for explaining concept of multiple functions

```
#include<stdio.h>
void First (void){
    printf("I am now inside function First\n");
}
void Second (void){
    printf("I am now inside function Second\n");
    First();
    printf("Back to Second\n");
}
int main (){
    printf("I am starting in function main\n");
    First ();
    printf("Back to main function \n");
    Second ();
    printf("Back to main function \n");
    return 0;
}
```

Lab Exercises

1. Write a function **Largest** to find the maximum of a given list of numbers. Also write a main program to read N numbers and find the largest among them using this function.
2. Input string & toggle the case of every character.
3. Write a function to find the factorial.
4. Arrange 'n' names in alphabetical order (hint: use string handling function-strcpy)

Additional Exercises

With the knowledge of modularization, function definition, function call etc., write C programs which implement simple functions.

Write C programs as specified below:

1. Write a function **Fact** to find the factorial of a given number. Using this function,

compute **NCR** in the main function.

2. Write a function **Largest** to find the maximum of a given list of numbers. Also write a main program to read N numbers and find the largest among them using this function.
3. Write a function **IsPalin** to check whether the given string is a palindrome or not. Write a main function to test this function.
4. Write a function **CornerSum** which takes as a parameter, no. of rows and no. of columns of a matrix and returns the sum of the elements in the four corners of the matrix. Write a main function to test the function.

LAB No.11

MODULAR PROGRAMMING-STRUCTURES

Objectives:

In this lab, student will be able to:

1. To Learn the concept of recursion and to write recursive functions
2. Write basic operations and programs using structures

Introduction:

- A recursive function is a function that invokes/calls itself directly or indirectly.
- A structure in the C programming language (and many derivatives) is a composite data type (or record) declaration that defines a physically grouped list of variables to be placed under one name in a block of memory.

Steps to Design a Recursive Algorithm

- **Base case:**
 - for a small value of n , it can be solved directly
- **Recursive case(s)**
 - Smaller versions of the same problem
- Algorithmic steps:
 - Identify the base case and provide a solution to it
 - Reduce the problem to smaller versions of itself
 - Move towards the base case using smaller versions

Solved Exercise

Program to explain the concept of recursive functions

```
#include<stdio.h>
long factorial (long a) {
    if (a ==0) //base case
        return (1);
    return (a * factorial (a-1));
}
int main () {
    long number;
    printf("Please type a number: ");
    scanf("%d",&number);
    printf("%d factorial is %ld",number, factorial (number));
    return 0;
}
```

Declaring and initializing pointers:

Syntax:

```
data_type * pt_name;
```

This tells the compiler 3 things about the pt_name:

- The asterisk (*) tells the variable pt_name is a pointer variable.
- pt_name needs a memory location.
- pt_name points to a variable of type data_type

Solved Exercise:

Declaration and initialization of structures:

Déclaration :

```
struct student
```

```
{    int rollno, age;
    char name[20];
}s1, s2, s3;
```

Initialisation :

```
int main( ){
    struct
    {    int rollno;
        int age;
    }stud={20, 21};
    ...
    ...
    return 0;
}
```

Solved Exercise:

```
#include<stdio.h>
struct Book{                //Structure Definition
char title[20];
char author[15];
int pages;
float price;
```

```

};
int main( ){
struct Book b[10];
int i,j;
printf("Input values");
for (i=0;i<10;i++){
scanf("%s %s %d %f",b[i].title,b[i].author,&b[i].pages,&b[i].price);
}
for (j=0;j<10;j++){
printf("title %s\n author %s\n pages %d\n
price%f\n",b[j].title,b[j].author,b[j].pages,b[j].price);
}
return 0;
}

```

Lab Exercises

Write C programs as specified below:

1. Create a student record with name, rollno, marks of 3 subjects (m1, m2, m3). Compute the average of marks for 3 students and display the names of the students in ascending order of their average marks.
2. Create an employee record with emp-no, name, age, date-of-joining (year), and salary. If there is 20% hike on salary per annum, compute the retirement year of each employee and the salary at that time. [standard age of retirement is 55]

LAB No.12

MODULAR PROGRAMMING-POINTERS

Objectives:

In this lab, student will be able to:

1. Declare and initialize pointer variable
2. Access a variable through its pointer

Introduction:

- A recursive function is a function that invokes/calls itself directly or indirectly.
- A Pointer is a memory location or a variable which stores the address of another variable in memory

Steps to Design a Recursive Algorithm

- **Base case:**
 - for a small value of n , it can be solved directly
- **Recursive case(s)**
 - Smaller versions of the same problem
- Algorithmic steps:
 - Identify the base case and provide a solution to it
 - Reduce the problem to smaller versions of itself
 - Move towards the base case using smaller versions

Solved Exercise

Declaring and initializing pointers:

Syntax:

`data_type * pt_name;`

This tells the compiler 3 things about the `pt_name`:

- The asterisk (*) tells the variable `pt_name` is a pointer variable.
- `pt_name` needs a memory location.
- `pt_name` points to a variable of type `data_type`

Solved Exercise:

```
#include<stdio.h>
int main()
{
    int var1 = 11; //two integer variables
    int var2 = 22;

    int *ptr;      //pointer to integer
    ptr = &var1;   //pointer points to var1
    printf("%d",*ptr); //print contents of pointer (11)
    ptr = &var2;   //pointer points to var2
    printf("%d",*ptr); //print contents of pointer (22)
    return 0;
}
```

Lab Exercises

1. Find out the length of string using pointer.
2. Write a recursive function, GCD to find the GCD of two numbers. Write a main program which reads 2 numbers and finds the GCD of the numbers using the specified function.
Ex: GCD of 9, 24 is 3.
3. Write a recursive function FIB to generate nth Fibonacci term. Write a main program to print first N Fibonacci terms using function FIB. [Hint: Fibonacci series is 0, 1, 1, 2, 3, 5, 8 ...]

Self-Learning Exercises

1. With respect to file handling practise the following functions:
(i) fopen()
(ii) fclose()
(iii) getc()
(iv) putc()
2. Write a C program to read a text file and count the number of lines, number of tabs, number of characters and number of blanks in it.

REFERENCES

1. E. Balaguruswamy, “Computing Fundamentals & C Programming”, Tata McGraw Hill, 2008.
2. E. Balaguruswamy, “Object Oriented Programming with C”, Tata McGraw Hill, 2nd Edition 2007.
3. Yashavant Kanetkar, “Let Us C”, 10th Edition, BPB Publications, 2010.
4. Dr. B. S. Grewal, “Numerical Methods in Engineering & Science”, Khanna publishers, 9th Edition, ISBN: 978-81-7409-248-9, 2010, 8th reprint, 2013.
5. Rudra Pratap, “Getting started with MATLAB – A Quick Introduction for Scientists and Engineers”, Oxford University Press, ISBN-13:978-0-19-806919-5, 2013.

C LANGUAGE QUICK REFERENCE

PREPROCESSOR

```
// Comment to end of line
/* Multi-line comment */
#include <stdio.h>      // Insert standard header file
#include "myfile.h"     // Insert file in current directory
#define X some text    // Replace X with some text
#define F(a,b) a+b      // Replace F(1,2) with 1+2
#define X \
    some text          // Line continuation
#undef X                // Remove definition
#ifdef(X)               // Conditional compilation (#ifdef X)
#else                   // Optional (#ifndef X or #if !defined(X))
#endif                 // Required after #if, #ifdef
```

LITERALS

```
255, 0377, 0xff        // Integers (decimal, octal, hex)
2147463647L, 0x7fffffff // Long (32-bit) integers
123.0, 1.23e2           // double (real) numbers
'a', '\141', '\x61'     // Character (literal, octal, hex)
'\n', '\\', '\'', '\"', // Newline, backslash, single quote, double quote
"string\n"              // Array of characters ending with newline and \0
"hello" "world"         // Concatenated strings
true, false             // bool constants 1 and 0
```

DECLARATIONS

```
int x;                  // Declare x to be an integer (value undefined)
int x=255;               // Declare and initialize x to 255
short s; long l;        // Usually 16 or 32 bit integer (int may be either)
char c= 'a';            // Usually 8 bit character
unsigned char u=255; signed char m=-1; // char might be either
unsigned long x=0xffffffffL; // short, int, long are signed
float f; double d;      // Single or double precision real (never unsigned)
```

```

bool b=true;           // true or false, may also use int (1 or 0)
int a, b, c;           // Multiple declarations
int a[10];              // Array of 10 ints (a[0] through a[9])
int a[]={0,1,2};        // Initialized array (or a[3]={0,1,2}; )
int a[2][3]={ {1,2,3},{4,5,6}}; // Array of array of ints
char s[]="hello";       // String (6 elements including '\0')
int* p;                 // p is a pointer to (address of) int
char* s="hello";         // s points to unnamed array containing "hello"
void* p=NULL;           // Address of untyped memory (NULL is 0)
int& r=x;               // r is a reference to (alias of) int x
enum weekend {SAT, SUN}; // weekend is a type with values SAT and SUN
enum weekend day;        // day is a variable of type weekend
enum weekend {SAT=0,SUN=1}; // Explicit representation as int
enum {SAT,SUN} day;     // Anonymous enum
typedef String char*;    // String s; means char* s;
const int c=3;          // Constants must be initialized, cannot assign
const int* p=a;         // Contents of p (elements of a) are constant
int* const p=a;         // p (but not contents) are constant
const int* const p=a;   // Both p and its contents are constant
const int& cr=x;        // cr cannot be assigned to change x

```

STORAGE CLASSES

```

int x;                 // Auto (memory exists only while in scope)
static int x;          // Global lifetime even if local scope
extern int x;          // Information only, declared elsewhere

```

STATEMENTS

```

x=y;                  // Every expression is a statement
int x;                // Declarations are statements
;                     // Empty statement
{                     // A block is a single statement
    int x;            // Scope of x is from declaration to end of
block
    a;                // In C, declarations must precede statements

```

```

}
if (x) a;           // If x is true (not 0), evaluate a
else if (y) b;      // If not x and y (optional, may be repeated)
else c;             // If not x and not y (optional)
while (x) a;        // Repeat 0 or more times while x is true
for (x; y; z) a;     // Equivalent to: x; while(y) {a; z;}
do a; while (x);     // Equivalent to: a; while(x) a;
switch (x) {         // x must be int
    case X1: a;      // If x == X1 (must be a const), jump here
    case X2: b;      // Else if x == X2, jump here
    default: c;      // Else jump here (optional)
}
break;              // Jump out of while, do, for loop, or switch
continue;           // Jump to bottom of while, do, or for loop
return x;           // Return x from function to caller
try { a; }
catch (T t) { b; }   // If a throws T, then jump here
catch (...) { c; }   // If a throws something else, jump here

```

FUNCTIONS

```

int f(int x, int);    // f is a function taking 2 ints and returning int
void f();             // f is a procedure taking no arguments
void f(int a=0);      // f() is equivalent to f(0)
f();                  // Default return type is int
inline f();           // Optimize for speed
f( ) { statements; }  // Function definition (must be global)

```

Function parameters and return values may be of any type. A function must either be declared or defined before it is used. It may be declared first and defined later. Every program consists of a set of global variable declarations and a set of function definitions (possibly in separate files), one of which must be:

```

int main() { statements... }    or
int main(int argc, char* argv[]) { statements... }

```

argv is an array of argc strings from the command line. By convention, main returns status 0 if successful, 1 or higher for errors.

EXPRESSIONS

Operators are grouped by precedence, highest first. Unary operators and assignment evaluate right to left. All others are left to right. Precedence does not affect order of evaluation which is undefined. There are no runtime checks for arrays out of bounds, invalid pointers etc.

T::X	// Name X defined in class T
N::X	// Name X defined in namespace N
::X	// Global name X
t.x	// Member x of struct or class t
p → x	// Member x of struct or class pointed to by p
a[i]	// i'th element of array a
f(x, y)	// Call to function f with arguments x and y
T(x, y)	// Object of class T initialized with x and y
x++	// Add 1 to x, evaluates to original x (postfix)
x--	// Subtract 1 from x, evaluates to original x
sizeof x	// Number of bytes used to represent object x
sizeof(T)	// Number of bytes to represent type T
++x	// Add 1 to x, evaluates to new value (prefix)
--x	// Subtract 1 from x, evaluates to new value
~x	// Bitwise complement of x
!x	// true if x is 0, else false (1 or 0 in C)
-x	// Unary minus
+x	// Unary plus (default)
&x	// Address of x
p	// Contents of address p (&x equals x)
x * y	// Multiply
x / y	// Divide (integers round toward 0)
x % y	// Modulo (result has sign of x)
x + y	// Add, or &x[y]
x - y	// Subtract, or number of elements from *x to *y
x << y	// x shifted y bits to left (x * pow(2, y))
x >> y	// x shifted y bits to right (x / pow(2, y))
x < y	// Less than

<code>x <= y</code>	<code>// Less than or equal to</code>
<code>x > y</code>	<code>// Greater than</code>
<code>x >= y</code>	<code>// Greater than or equal to</code>
<code>x == y</code>	<code>// Equals</code>
<code>x != y</code>	<code>// Not equals</code>
<code>x & y</code>	<code>// Bitwise and (3 & 6 is 2)</code>
<code>x ^ y</code>	<code>// Bitwise exclusive or (3 ^ 6 is 5)</code>
<code>x y</code>	<code>// Bitwise or (3 6 is 7)</code>
<code>x && y</code>	<code>// x and then y (evaluates y only if x (not 0))</code>
<code>x r</code>	<code>// x or else y (evaluates y only if x is false(0))</code>
<code>x = y</code>	<code>// Assign y to x, returns new value of x</code>
<code>x += y</code>	<code>// x = x + y, also -= *= /= <<= >>= &= = ^=</code>
<code>x ? y : z</code>	<code>// y if x is true (nonzero), else z</code>
<code>throw x</code>	<code>// Throw exception, aborts if not caught</code>
<code>x, y</code>	<code>// evaluates x and y, returns y (seldom used)</code>

STDIO.H.H. STDIO.H

<code>cin >> x >> y;</code>	<code>// Read words x and y (any type) from stdin</code>
<code>cout << "x=" << 3 << endl;</code>	<code>// Write line to stdout</code>
<code>cerr << x << y << flush;</code>	<code>// Write to stderr and flush</code>
<code>c = cin.get();</code>	<code>// c = getchar();</code>
<code>cin.get(c);</code>	<code>// Read char</code>
<code>cin.getline(s, n, '\n');</code>	<code>// Read line into char s[n] to '\n', (default)</code>
<code>if (cin)</code>	<code>// Good state (not EOY)?</code>
<code>// To read/write any type T:</code>	

STRING (Variable sized character array)

<code>string s1, s2= "hello";</code>	<code>//Create strings</code>
<code>s1.size(), s2.size();</code>	<code>// Number of characters: 0, 5</code>
<code>s1 += s2 + ' ' + "world";</code>	<code>// Concatenation</code>
<code>s1 == "hello world";</code>	<code>// Comparison, also <, >, !=, etc.</code>
<code>s1[0];</code>	<code>// 'h'</code>
<code>s1.substr(m, n);</code>	<code>// Substring of size n starting at s1[m]</code>

<code>sl.c_str();</code>	<code>// Convert to const char*</code>
<code>getline(cin, s);</code>	<code>// Read line ending in '\n'</code>
<code>asin(x); acos(x); atan(x);</code>	<code>// Inverses</code>
<code>atan2(y, x);</code>	<code>// atan(y/x)</code>
<code>sinh(x); cosh(x); tanh(x);</code>	<code>// Hyperbolic</code>
<code>exp(x); log(x); log10(x);</code>	<code>// e to the x, log base e, log base 10</code>
<code>pow(x, y); sqrt(x);</code>	<code>// x to the y, square root</code>
<code>ceil(x); floor(x);</code>	<code>// Round up or down (as a double)</code>
<code>fabs(x); fmod(x, y);</code>	<code>// Absolute value, x mod y</code>

EXERCISE SOLUTIONS

C program to find all roots of a quadratic equation using switch case:

Quadratic equation

In elementary algebra quadratic equation is an equation in the form of

$$ax^2 + bx + c = 0$$

Solving quadratic equation

A quadratic equation can have either one or two distinct real or complex roots depending upon nature of discriminant of the equation. Where discriminant of the quadratic equation is given by

$$\Delta = b^2 - 4ac$$

Depending upon the nature of the discriminant, formula for finding roots can be given as:

- Case 1: If **discriminant is positive**. Then there are two real distinct roots given by.

$$\frac{-b + \sqrt{\Delta}}{2a} \quad \text{and} \quad \frac{-b - \sqrt{\Delta}}{2a}$$

- Case 2: If **discriminant is zero**. Then it have exactly one real root given by.

$$-\frac{b}{2a}$$

- Case 3: If **discriminant is negative**. Then it will have two distinct complex roots given by.

$$\frac{-b}{2a} + i \frac{\sqrt{-\Delta}}{2a} \quad \text{and} \quad \frac{-b}{2a} - i \frac{\sqrt{-\Delta}}{2a}$$

Let see an example,

Input : a = 1, b = -2, c = 1

Output: Roots are real and same

1

Input : a = 1, b = 7, c = 12

Output: Roots are real and different

-3, -4

Input : a = 1, b = 1, c = 1

Output : Roots are complex

-0.5 + i1.73205

-0.5 - i1.73205

Quadratic equation:

In algebra, a quadratic equation is an equation that can be rearranged in standard form as,
 $ax^2 + bx + c = 0$

Below is a direct formula for finding the roots of the quadratic equation.

There are the following important cases of this formula.

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Case 1: (Discriminant < 0)

$b^2 < 4ac$, then roots are complex(not real).

For example roots of $x^2 + x + 1$, roots are

-0.5 + i1.73205 and -0.5 - i1.73205

Case 2: (Discriminant == 0)

$b^2 == 4ac$, then roots are real and both roots are the same.

For example, roots of $x^2 - 2x + 1$ are 1 and 1

Case 3: (Discriminant > 0)

$b^2 > 4ac$, then roots are real and different.

For example, roots of $x^2 - 7x - 12$ are 3 and 4

Logic to find roots of quadratic equation using switch case:

- Ask the user to enter the value of a, b, and C.
- Calculate the discriminant value using the a, b, and c.
- Compute the roots based on the nature of the discriminant.
- The expression (discriminant > 0) can have two possible cases i.e. case 0 and case 1.
- For case 1 (when discriminant +ve) compute the roots as per the above-mentioned formula.
- For case 0, again there are two scenario discriminant is -ve (discriminant < 0) or discriminant is zero (discriminant == 0).
- The below program ask the user to enter the value of a,b and c. After getting the value from the user it will calculate on the basis of 'Discriminant' value using switch case.

```
• #include <stdio.h>
• #include <math.h>
• int main()
• {
•     float a, b, c;
•     float root1, root2, imaginary, discriminant;
•     printf("Enter value of 'a' of quadratic equation (aX^2 + bX + c): ");
•     scanf("%f", &a);
•     printf("Enter value of 'b' of quadratic equation (aX^2 + bX + c): ");
•     scanf("%f", &b);
•     printf("Enter values of 'c' of quadratic equation (aX^2 + bX + c): ");
•     scanf("%f", &c);
•     // Find discriminant of the equation
•     discriminant = (b * b) - (4 * a * c);
•     switch(discriminant > 0)
•     {
•     case 1:
•         // If discriminant is positive
•         root1 = (-b + sqrt(discriminant)) / (2 * a);
```

- `root2 = (-b - sqrt(discriminant)) / (2 * a);`
- `printf("Two distinct and real roots exists: %.2f and %.2f",`
- `root1, root2);`
- `break;`
- `case 0:`
- `// If discriminant is not positive`
- `switch(discriminant < 0)`
- `{`
- `case 1:`
- `// If discriminant is negative`
- `root1 = root2 = -b / (2 * a);`
- `imaginary = sqrt(-discriminant) / (2 * a);`
- `printf("Two distinct complex roots exists: %.2f + i%.2f and %.2f - i%.2f",`
- `root1, imaginary, root2, imaginary);`
- `break;`
- `case 0:`
- `// If discriminant is zero`
- `root1 = root2 = -b / (2 * a);`
- `printf("Two equal and real roots exists: %.2f and %.2f", root1, root2);`
- `break;`
- `}`
- `}`
- `return 0;`
- `}`
- **Output:**
- Enter value of **a** of quadratic equation ($aX^2 + bX + c$): 2
- Enter value of **b** of quadratic equation ($aX^2 + bX + c$): 7
- Enter values of **c** of quadratic equation ($aX^2 + bX + c$): 2
- Two distinct and real roots exist: -0.31 and -3.19

Version 2

Example

Input

Input a: 4

Input b: -2

Input c: -10

Output

Root1: 1.85

Root2: -1.35

2. C program to find out smallest number out of 3 numbers using conditional operator

Conditional Operator in C

The conditional operator is also known as a **ternary operator**. The conditional statements are the decision-making statements which depends upon the output of the expression. It is represented by two symbols, i.e., '?' and ':'.

As conditional operator works on three operands, so it is also known as the ternary operator.

The behaviour of the conditional operator is similar to the 'if-else' statement as 'if-else' statement is also a decision-making statement.

The conditional operator is kind of similar to the if-else statement as it does follow the same algorithm as of if-else statement but the conditional operator takes less space and helps to write the if-else statements in the shortest way possible.

Syntax of a conditional operator

Syntax:

The conditional operator is of the form

variable = Expression1 ? Expression2 : Expression3

Or the syntax will also be in this form

Syntax of Conditional Operator in C

```
expression1 ? expression2 : expression3;
```

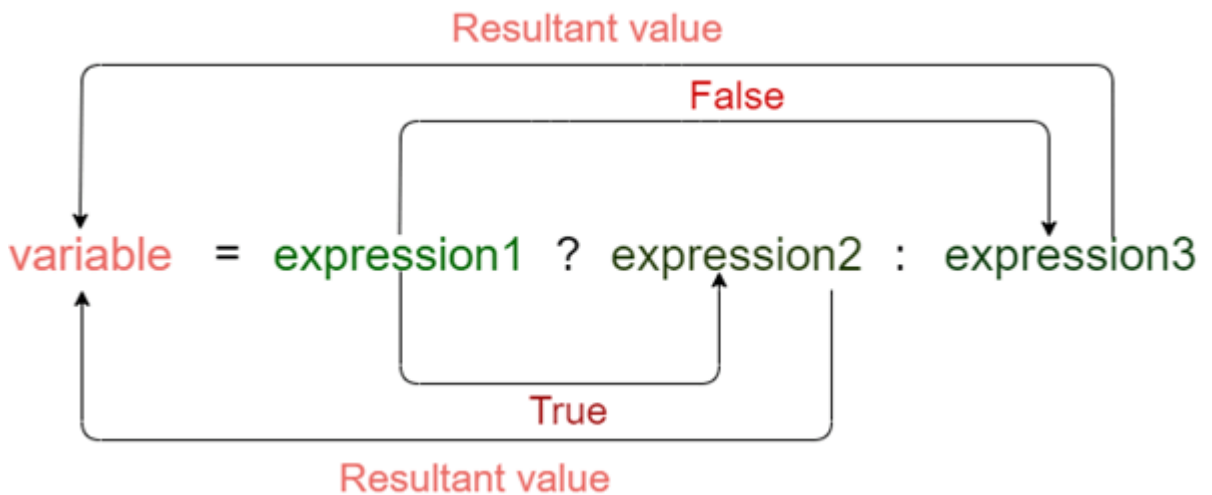
or for simplicity, we write it as

```
condition ? true-statement : false-statement;
```

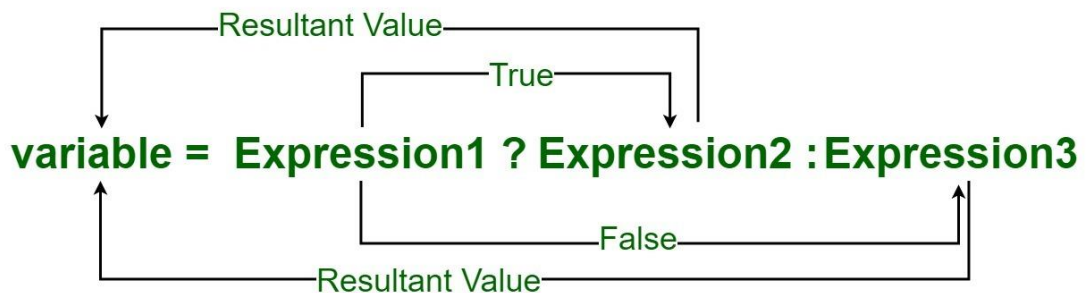
The expression1 is evaluated, it is treated as a **logical condition**. If the result is non-zero then expression2 will be evaluated otherwise expression3 will be evaluated. The value after evaluation of expression2 or expression3 is the final result.

or

1. Expression1? expression2: expression3;



Conditional or Ternary Operator (?:) in C/C++



variable = (condition) ? Expression2 : Expression3

Or syntax will also be in this form

(condition) ? (variable = Expression2) : (variable = Expression3)

It can be visualized into if-else statement as:

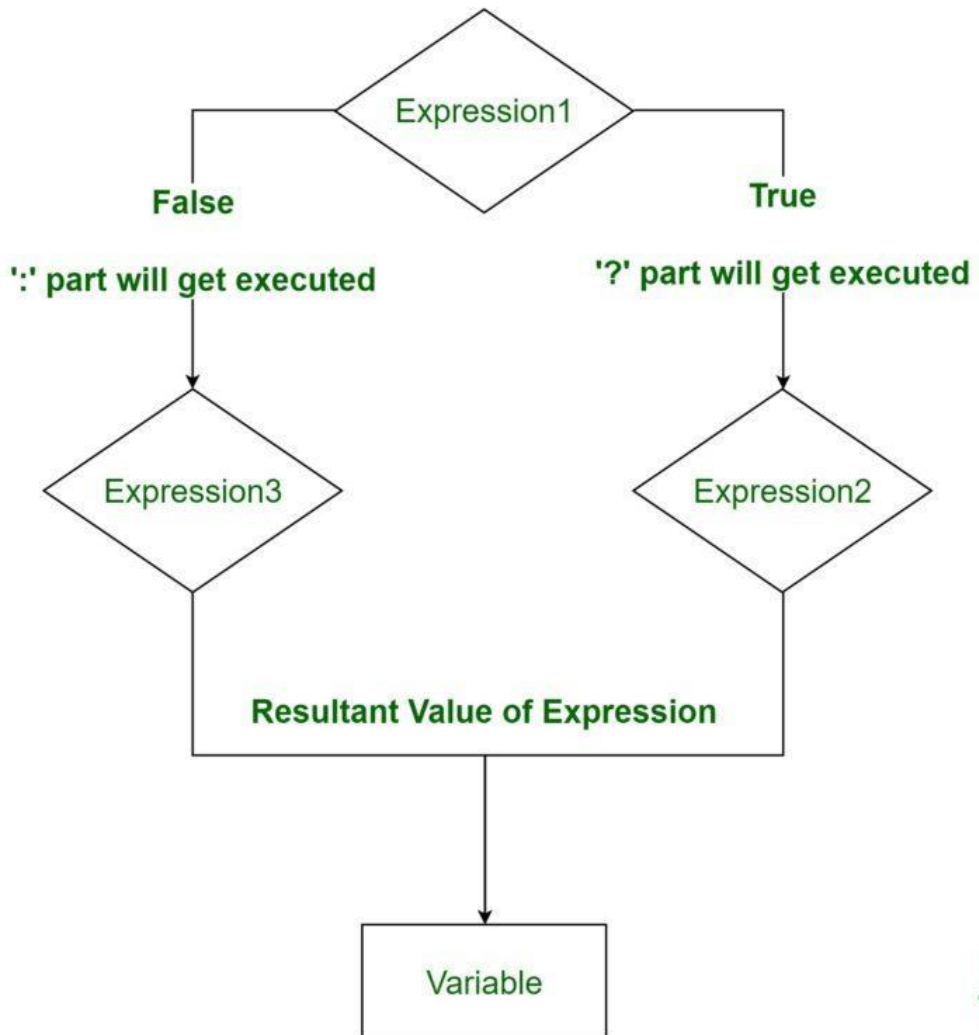
```
if(Expression1)
{
    variable = Expression2;
}
else
{
    variable = Expression3;
}
```

Since the Conditional Operator ‘?:’ takes three operands to work, hence they are also called **ternary operators**.

Working:

Here, **Expression1** is the condition to be evaluated. If the condition(**Expression1**) is True then **Expression2** will be executed and the result will be returned. Otherwise, if the condition(**Expression1**) is false then **Expression3** will be executed and the result will be returned.

Flow Chart of Conditional or Ternary Operator

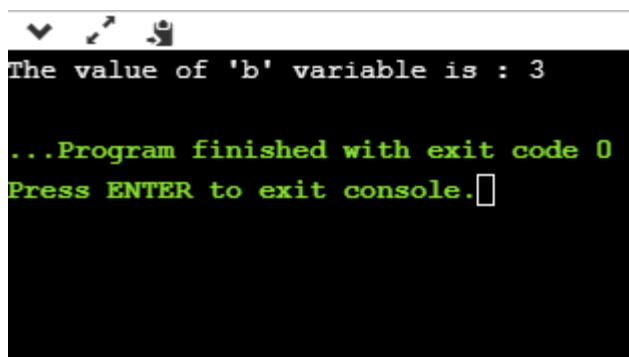


Meaning of the above syntax.

- In the above syntax, the expression1 is a Boolean condition that can be either true or false value.
- If the expression1 results into a true value, then the expression2 will execute.
- The expression2 is said to be true only when it returns a non-zero value.
- If the expression1 returns false value then the expression3 will execute.
- The expression3 is said to be false only when it returns zero value.
- `#include <stdio.h>`
- `int main()`
- `{`
- `int a=5,b; // variable declaration`
- `b=((a==5)?(3):(2)); // conditional operator`
- `printf("The value of 'b' variable is : %d",b);`
- `return 0;`
- `}`

In the above code, we have declared two variables, i.e., 'a' and 'b', and assign 5 value to the 'a' variable. After the declaration, we are assigning value to the 'b' variable by using the conditional operator. If the value of 'a' is equal to 5 then 'b' is assigned with a 3 value otherwise 2.

Output



```
The value of 'b' variable is : 3

...Program finished with exit code 0
Press ENTER to exit console.
```

- A conditional operator is a single programming statement, while the 'if-else' statement is a programming block in which statements come under the parenthesis.
- A conditional operator can also be used for assigning a value to the variable, whereas the 'if-else' statement cannot be used for the assignment purpose.
- It is not useful for executing the statements when the statements are multiple, whereas the 'if-else' statement proves more suitable when executing multiple statements.
- The nested ternary operator is more complex and cannot be easily debugged, while the nested 'if-else' statement is easy to read and maintain.

C program to find out smallest number out of 3 numbers using conditional operator

```

○ #include<stdio.h>
○
○ void main()
○ {
○     int a, b, c;
○
○     printf("Enter three numbers\n");
○     scanf("%d %d %d", &a, &b, &c);
○
○     ((a < b) && (a < c)) ? printf("a is smallest\n") :
○     (b < c) ? printf("b is smallest\n") :
○     printf("c is smallest\n");
○ }

```

○ *Output 1*

```

○ Enter three numbers
○ 10
○ 24
○ 65
○ a is smallest
○

```

- **Output 2**
-
- Enter three numbers
- 45
- 31
- 65
- b is smallest

- **Output 3**
-
- Enter three numbers
- 78
- 54
- 31
- c is smallest

2. Same program using if else statement

Find Smallest of Three Numbers in C

Here is the program that will find out the smallest of three numbers provided by user at run-time.

```
#include<stdio.h>
#include<conio.h>
int main()
{
    int a, b, c, small;
    printf("Enter three numbers : ");
    scanf("%d%d%d", &a, &b, &c);
    if(a<b)
    {
        if(b<c)
            small = a;
        else
        {
            if(a<c)
                small = a;
            else
```

```

        small = c;
    }
}
else
{
    if(b<c)
        small = b;
    else
        small = c;
}
printf("\nSmallest number is: %d", small);
getch();
return 0;
}

```

3. / C program to find largest among two
// numbers using ternary operator

```

#include <stdio.h>

int main()
{
    int m = 5, n = 4;

    (m > n) ? printf("m is greater than n that is %d > %d",
        m, n)
        : printf("n is greater than m that is %d > %d",
        n, m);

    return 0;
}

```

3.Find the largest among given 3 numbers

```

#include<stdio.h>
int main()
{
    float a, b, c, max;
    printf("Enter three numbers: ");
    scanf("%f %f %f",&a, &b, &c);
}

```

```

max = (a>b && a>c)? a: (b>a && b>c)? b:c;
printf("Maximum number = %.2f",max);
return 0;
}

```

Example 1: Using if Statement

```

#include <stdio.h>

int main() {

    double n1, n2, n3;

    printf("Enter three different numbers: ");
    scanf("%lf %lf %lf", &n1, &n2, &n3);

    // if n1 is greater than both n2 and n3, n1 is the largest
    if (n1 >= n2 && n1 >= n3)
        printf("%.2f is the largest number.", n1);

    // if n2 is greater than both n1 and n3, n2 is the largest
    if (n2 >= n1 && n2 >= n3)
        printf("%.2f is the largest number.", n2);

    // if n3 is greater than both n1 and n2, n3 is the largest
    if (n3 >= n1 && n3 >= n2)
        printf("%.2f is the largest number.", n3);

    return 0;
}

```

Using if...else Ladder

```

#include <stdio.h>

int main() {

```

```

double n1, n2, n3;

printf("Enter three numbers: ");
scanf("%lf %lf %lf", &n1, &n2, &n3);

// if n1 is greater than both n2 and n3, n1 is the largest
if (n1 >= n2 && n1 >= n3)
    printf("%.2lf is the largest number.", n1);

// if n2 is greater than both n1 and n3, n2 is the largest
else if (n2 >= n1 && n2 >= n3)
    printf("%.2lf is the largest number.", n2);

// if both above conditions are false, n3 is the largest
else
    printf("%.2lf is the largest number.", n3);

return 0;
}

```

[Run Code](#)

In this program, only the `if` statement is executed when `n1` is the largest.

Similarly, only the `else if` statement is executed when `n2` is the largest, and so on.

Example 3: Using Nested if...else

```

#include <stdio.h>

int main() {

    double n1, n2, n3;

```

```

printf("Enter three numbers: ");
scanf("%lf %lf %lf", &n1, &n2, &n3);

// outer if statement
if (n1 >= n2) {

    // inner if...else
    if (n1 >= n3)
        printf("%.2lf is the largest number.", n1);
    else
        printf("%.2lf is the largest number.", n3);
}

// outer else statement
else {

    // inner if...else
    if (n2 >= n3)
        printf("%.2lf is the largest number.", n2);
    else
        printf("%.2lf is the largest number.", n3);
}

return 0;
}

```

Write a C program to find the maximum in the given two numbers using the conditional operator.

```

#include<stdio.h>
int main()
{
    float num1, num2, max;

    printf("Enter two numbers: ");
    scanf("%f %f", &num1, &num2);
}

```

```

max = (num1 > num2) ? num1 : num2;

printf("Maximum of %.2f and %.2f = %.2f",
      num1, num2, max);

return 0;
}

```

Output:-

```

Enter two numbers: 12.5 10.5
Maximum of 12.50 and 10.50 = 12.50

```

- 4 Write a program that will read the value of x and evaluate the following function

$$Y = \begin{cases} 1 & \text{for } x > 0 \\ 0 & \text{for } x = 0 \\ -1 & \text{for } x < 0 \end{cases}$$

Use else if statements & Print the result ('Y' value).

```

#include <stdio.h>
#include <conio.h>
Void main()
{
    int x, y;
    printf("Enter the integer value for x \n");
    scanf("%d", &x);
    if(x==0)
    {
        printf("the value of y = 0, as the value of x = %d\n", x);
    }
    else if(x>0)
    {
        printf("the value of y = 1, as the value of x = %d\n", x);
    }
    else if(x<0)
    {
        printf("the value of y = -1, as the value of x = %d\n", x);
    }
}

```

Problem Description

This C program accepts an integer, reverse it and also checks if it is a palindrome or not.

Problem Solution

1. Take the number which you have to reverse as the input.
2. Obtain its quotient and remainder.
3. Multiply the separate variable with 10 and add the obtained remainder to it.
4. Do step 2 again for the quotient and step 3 for the remainder obtained in step 4.
5. Repeat the process until quotient becomes zero.
6. When it becomes zero, check if the reversed number is equal to original number or not.
7. Print the output and exit.

Program/Source Code

Here is source code of the C program to reverse a number & checks it is a palindrome or not.

```
1.
2.
3. #include <stdio.h>
4.
5. void main()
6. {
7.     int num, temp, remainder, reverse = 0;
8.
9.     printf("Enter an integer \n");
10.    scanf("%d", &num);
11.    /* original number is stored at temp */
12.    temp = num;
13.    while (num > 0)
14.    {
15.        remainder = num % 10;
16.        reverse = reverse * 10 + remainder;
17.        num /= 10;
18.    }
19.    printf("Given number is = %d\n", temp);
```

```

20. printf("Its reverse is = %d\n", reverse);
21. if (temp == reverse)
22.     printf("Number is a palindrome \n");
23. else
24.     printf("Number is not a palindrome \n");
25. }

```

Program Explanation

1. Take the number which you have to reverse as the input and store it in the variable num.
2. Copy the input number to the another variable temp.
3. Firstly initialize the variable reverse to zero.
4. Obtain the remainder of the input number.
5. Multiply the variable reverse with 10 and add the Obtained remainder to it and store the result in the same variable.
6. Obtain the quotient of the input number and considering this as input number repeat the steps as mentioned above until the obtained quotient becomes zero.
7. When it becomes zero, using if,else statement check whether the reversed number is equal to original number or not.
8. If it is equal, then print the output as “Number is a palindrome”, otherwise print the output as “Number is not a palindrome”.

Output

When the above program is executed, it produces the following result –

```

enter a number:
1234
reversed number is = 4321
1234 is not the Palindrome Number.
enter a number:
1221
reversed number is = 1221
1221 is Palindrome Number.

```

Output:

```
enter the number=151
```

palindrome number

enter the number=5621

not palindrome number

2. Generate prime numbers between 2 given limits.(use while loop)

```
1. #include < stdio.h >
2. #include < math.h >
3.
4. int main()
5. {
6.     int start, end, count, prime, inum;
7.
8.     printf("Enter start and end value\n");
9.     scanf("%d%d", &start, &end);
10.
11.     printf("\n\nPrime Numbers from %d to %d are:\n", start, end);
12.     while(start <= end)
13.     {
14.         inum = sqrt(start);
15.         count = 2;
16.         prime = 1;
17.
18.         while(count <= inum)
19.         {
20.             if(start%count == 0)
21.             {
22.                 prime = 0;
23.                 break;
24.             }
25.             count++;
26.         }
27.
28.         if(prime) printf("%d, ", start);
29.         start++;
30.     }
31.     printf("\n\n");
```

```
32.  
33.     return 0;  
34. }
```

Output 1:

Enter start and end value

10

50

Prime Numbers from 10 to 50 are:

11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47,

Output 2:

Enter start and end value

50

100

Prime Numbers from 50 to 100 are:

53, 59, 61, 67, 71, 73, 79, 83, 89, 97,

Version 2 Using Go to jump

```
#include<stdio.h>
```

```
int main(){
```

```
    int numbr,k,remark,first,last;
```

```
    printf("Give the First Number for the Range : ");
```

```
    scanf("%d",&first);
```

```
    printf("\n Give the Last Number for the Range : ");
```

```
    scanf("%d",&last);
```

```
    numbr= first;
```

```

while(numbr<= last)
{
    remark=0;
    k=2;
    while(k<=numbr/2)
    {
        if((numbr % k) == 0){
            remark++;
            goto jump;
        }
        k++;
    }
    if(remark==0)
        printf("\n   %d ",numbr);
    jump:
    ++numbr;
}
return 0;
}

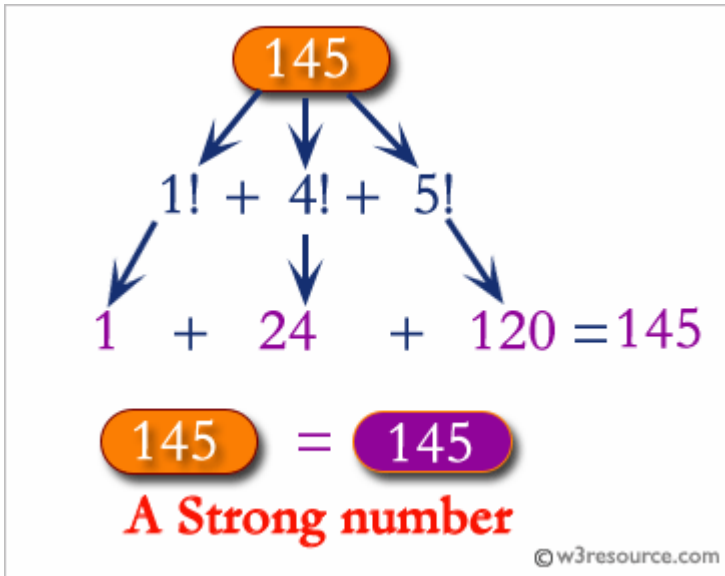
```

1. the C program to Check whether the number is strong number or not.

Write a to check whether a number is a Strong Number or not.

[Hint: Positive number whose sum of the factorial of its digits is equal to the number itself] Ex: $145 = 1! + 4! + 5! = 1 + 24 + 120 = 145$ is a strong number.

Pictorial Presentation:



2.

C Code:

Sample Solution:

```
/**
 * C program to check whether a number is Strong Number or not
 */

#include <stdio.h>

int main()
{
    int i, originalNum, num, lastDigit, sum;
    long fact;

    /* Input a number from user */
    printf("Enter any number to check Strong number: ");
    scanf("%d", &num);

    /* Copy the value of num to a temporary variable */
```

```

originalNum = num;

sum = 0;

/* Find sum of factorial of digits */
while(num > 0)
{
    /* Get last digit of num */
    lastDigit = num % 10;

    /* Find factorial of last digit */
    fact = 1;
    for(i=1; i<=lastDigit; i++)
    {
        fact = fact * i;
    }

    /* Add factorial to sum */
    sum = sum + fact;

    num = num / 10;
}

/* Check Strong number condition */
if(sum == originalNum)
{
    printf("%d is STRONG NUMBER", originalNum);
}
else
{
    printf("%d is NOT STRONG NUMBER", originalNum);
}

return 0;
}

```

Output

```

Enter any number to check Strong number: 145
145 is STRONG NUMBER

```

Copy

Sample Output:

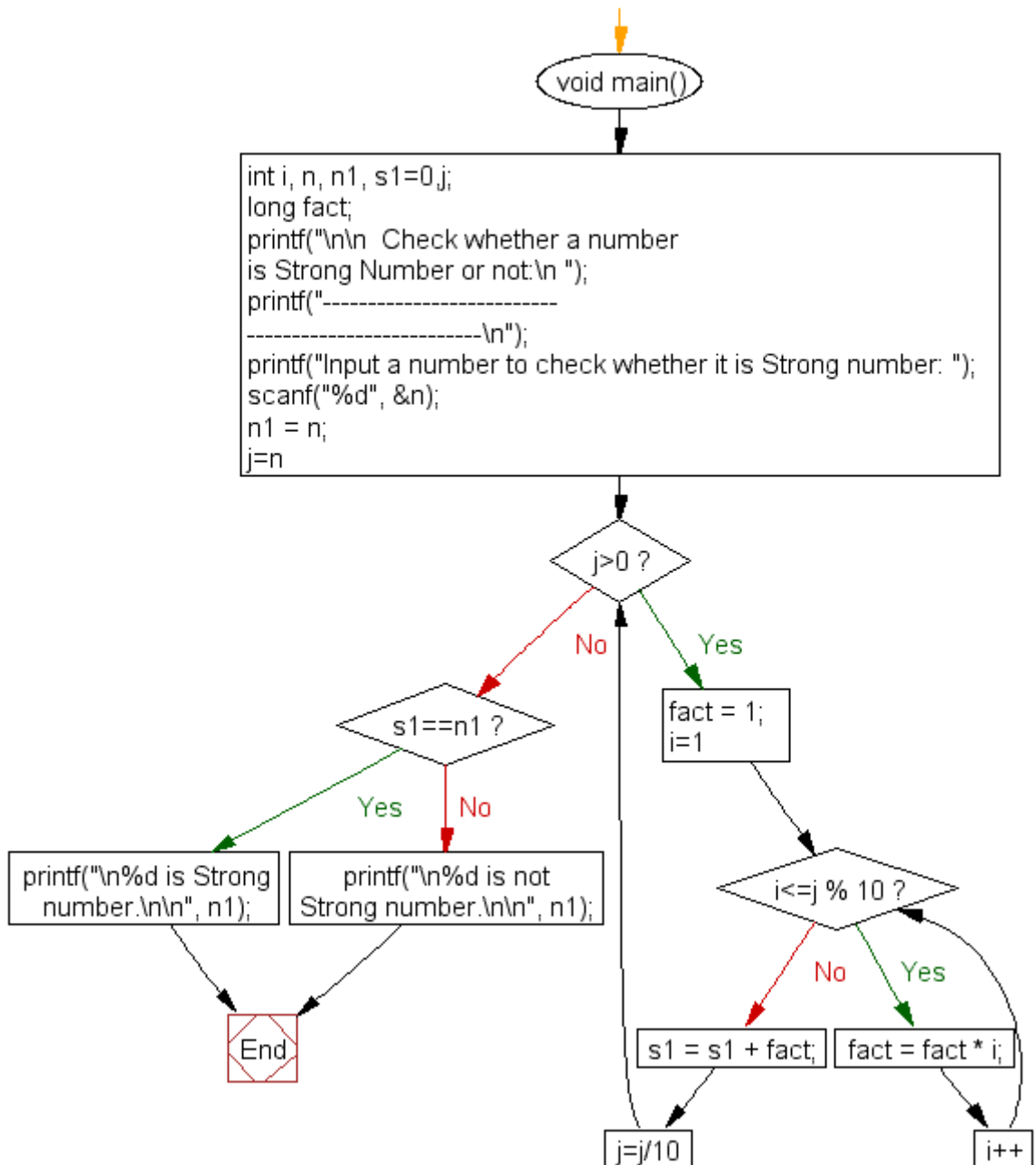
Check whether a number is Strong Number or not:

Input a number to check whether it is Strong number: 15

15 is not Strong number.

Flowchart:

5.6.



6..WACP check the number is Armstrong or not.

When the sum of the cube of the individual digits of a number is equal to that number, the

number is called Armstrong number. For Example 153 is an Armstrong number because $153 = 1^3 + 5^3 + 3^3$.

Test Data :

Input a number: 153

Expected Output :

153 is an Armstrong number.

Pictorial Presentation:

Armstrong Number :

Number = 153

$$1^3 + 5^3 + 3^3$$
$$1 + 125 + 27 = 153$$

Sum = Original Number

153 is Armstrong Number

© w3resource.com

Visualize C code execution:

```
#include <stdio.h>
```

```
void main(){
```

```
    int num,r,sum=0,temp;
```

```
    printf("Input a number: ");
```

```

scanf("%d",&num);

for(temp=num;num!=0;num=num/10){

    r=num % 10;

    sum=sum+(r*r*r);

}

if(sum==temp)

    printf("%d is an Armstrong number.\n",temp);

else

    printf("%d is not an Armstrong number.\n",temp);

}

```

Copy

Sample Output:

Input a number: 153
153 is an Armstrong number.

enter the number=371

This number is Armstrong number

Check: $371 = 3*3*3 + 7*7*7 + 1*1*1$

$= 27 + 343 + 1$

$= 371$

enter the number=53

This number is not Armstrong number

Explanation

$53 = 5*5*5 + 3*3*3$

```
=125 +27  
= 152 != 53
```

6. C program to Use break and continue keyword in while loop.

Break

You have already seen the **break** statement used in an earlier chapter of this tutorial. It was used to "jump out" of a **switch** statement.

The **break** statement can also be used to jump out of a **loop**.

This example jumps out of the loop when **i** is equal to 4:

Continue

The **continue** statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

This example skips the value of 4:

Break and Continue in While Loop

You can also use **break** and **continue** in while loops:

Break Example

```
int i = 0;  
  
while (i < 10) {  
    if (i == 4) {  
        break;  
    }  
    printf("%d\n", i);  
    i++;  
}
```

Continue Example

```

int i = 0;

while (i < 10) {
    if (i == 4) {
        i++;
        continue;
    }
    printf("%d\n", i);
    i++;
}

```

1. Generate the multiplication table for ' n ' numbers up to ' k ' terms (using nested for loops).

[Hint: 1 2 3 4 5 k

2 4 6 8 102*k

.....

n..... n*k]

Write a program in C to display the multiplication table vertically from 1 to n.

Pictorial Presentation:



Multiplication table from 1 to 12



	1	2	3	4	5	6	7	8	9	10	11	12
1	1	2	3	4	5	6	7	8	9	10	11	12
2	2	4	6	8	10	12	14	16	18	20	22	24
3	3	6	9	12	15	18	21	24	27	30	33	36
4	4	8	12	16	20	24	28	32	36	40	44	48
5	5	10	15	20	25	30	35	40	45	50	55	60
6	6	12	18	24	30	36	42	48	54	60	66	72
7	7	14	21	28	35	42	49	56	63	70	77	84
8	8	16	24	32	40	48	56	64	72	80	88	96
9	9	18	27	36	45	54	63	72	81	90	99	108
10	10	20	30	40	50	60	70	80	90	100	110	120
11	11	22	33	44	55	66	77	88	99	110	121	132
12	12	24	36	48	60	72	84	96	108	120	132	144

© w3resource.com

Sample Solution:

C Code:

```
#include <stdio.h>

int main()
```

```

{
    int j,i,n;

    printf("Input upto the table number starting from 1 : ");

    scanf("%d",&n);

    printf("Multiplication table from 1 to %d \n",n);

    for(i=1;i<=10;i++)
    {
        for(j=1;j<=n;j++)
        {

            printf("%dx%d = %d, ",j,i,i*j);

        }

        printf("\n");

    }
}

```

Copy

Sample Output:

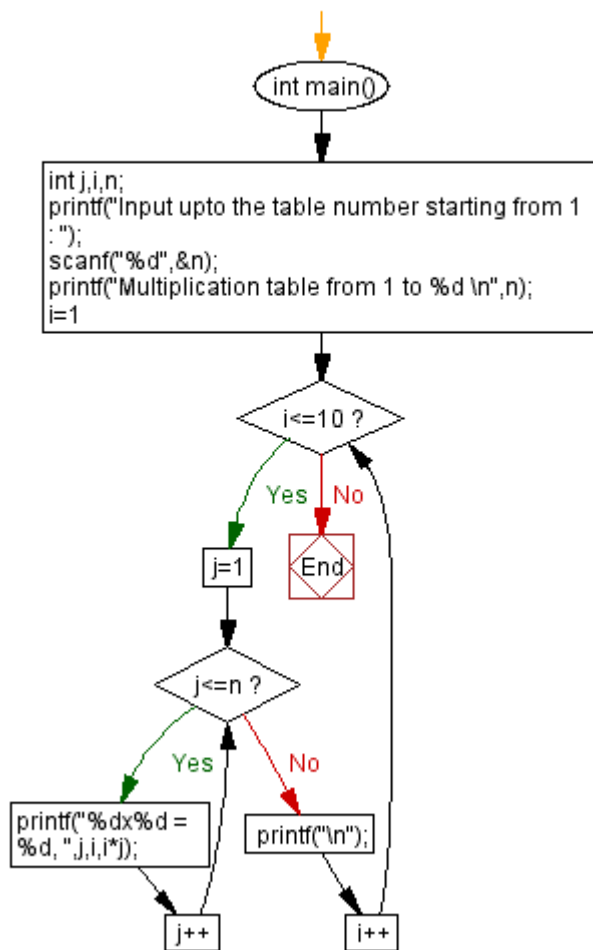
Input upto the table number starting from 1 : 8

Multiplication table from 1 to 8

1x1 = 1, 2x1 = 2, 3x1 = 3, 4x1 = 4, 5x1 = 5, 6x1 = 6, 7x1 = 7, 8x1 = 8
 1x2 = 2, 2x2 = 4, 3x2 = 6, 4x2 = 8, 5x2 = 10, 6x2 = 12, 7x2 = 14, 8x2 = 16
 1x3 = 3, 2x3 = 6, 3x3 = 9, 4x3 = 12, 5x3 = 15, 6x3 = 18, 7x3 = 21, 8x3 = 24
 1x4 = 4, 2x4 = 8, 3x4 = 12, 4x4 = 16, 5x4 = 20, 6x4 = 24, 7x4 = 28, 8x4 = 32
 1x5 = 5, 2x5 = 10, 3x5 = 15, 4x5 = 20, 5x5 = 25, 6x5 = 30, 7x5 = 35, 8x5 = 40
 1x6 = 6, 2x6 = 12, 3x6 = 18, 4x6 = 24, 5x6 = 30, 6x6 = 36, 7x6 = 42, 8x6 = 48

$1 \times 7 = 7, 2 \times 7 = 14, 3 \times 7 = 21, 4 \times 7 = 28, 5 \times 7 = 35, 6 \times 7 = 42, 7 \times 7 = 49, 8 \times 7 = 56$
 $1 \times 8 = 8, 2 \times 8 = 16, 3 \times 8 = 24, 4 \times 8 = 32, 5 \times 8 = 40, 6 \times 8 = 48, 7 \times 8 = 56, 8 \times 8 = 64$
 $1 \times 9 = 9, 2 \times 9 = 18, 3 \times 9 = 27, 4 \times 9 = 36, 5 \times 9 = 45, 6 \times 9 = 54, 7 \times 9 = 63, 8 \times 9 = 72$
 $1 \times 10 = 10, 2 \times 10 = 20, 3 \times 10 = 30, 4 \times 10 = 40, 5 \times 10 = 50, 6 \times 10 = 60, 7 \times 10 = 70, 8 \times 10 = 80$

Flowchart:



2. Generate Floyd's triangle using natural numbers for a given limit N. (using for loops)

[Hint: Floyd's triangle is a right angled-triangle using the natural numbers]

Ex: Input: N = 4 14

Output:

```
1
2 3
4 5 6
7 8 9 10
```

Algorithm

1. Create variables that hold rows and column values as i and j. Take a number to display the rows as num and set the variable k to 1 as its initial value.
2. Use nested for loops:
 - Outer for loop starts its iteration i = 1 up to n rows.
 - Inner for loop starts its iteration from j = 1 up to (j <= i).
3. Print the values of k.
4. Increment k by 1 or $k = k + 1$.
5. Jump to newline after each iteration of the inner for loop.
6. Stop

```
#include <stdio.h>

int main()
{
    int n, i, c, a = 1;

    printf("Enter the number of rows of Floyd's triangle to print\n");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        for (c = 1; c <= i; c++)
```

```

    {
        printf("%d ",a);
        a++;
    }
    printf("\n");
}

return 0;
}

```

floyd.c

```

1. #include <stdio.h>
2. #include <conio.h>
3. void main()
4. {
5.     int num, i, j, k = 1;
6.     printf( " Enter a number to define the rows in Floyd's triangle: \n");
7.     scanf( "%d", &num);
8.     // use nested for loop
9.     // outer for loop define the rows and check rows condition
10.    for (i = 1; i <= num; i++)
11.    {
12.        // inner loop check j should be less than equal to i and print the data.
13.        for (j = 1; j <= i; j++)
14.        {
15.            printf( " %2d", k++); // print the number
16.        }
17.        printf( "\n");
18.    }
19.    getch();

```

20. }

Output

Enter a number to define the rows in Floyd's triangle:

```
6
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
16 17 18 19 20 21
```

2nd time execution of the program:

Enter a number to define the rows in Floyd's triangle:

```
5
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

3. Evaluate the sine series, $\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$ to n terms.

$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} - \dots$

Source Code

```
#include <stdio.h>

int main()
{
    int i, j, n, fact, sign = - 1;
    float x, p, sum = 0;
    printf("Enter the value of x : ");
    scanf("%f", &x);
```

```

printf("Enter the value of n : ");
scanf("%d", &n);
for (i = 1; i <= n; i += 2)
{
    p = 1;
    fact = 1;
    for (j = 1; j <= i; j++)
    {
        p = p * x;
        fact = fact * j;
    }
    sign = - 1 * sign;
    sum += sign * p / fact;
}
printf("sin %0.2f = %f", x, sum);
return 0;
}

```

Output

```

Enter the value of x : 2
Enter the value of n : 3
sin 2.00 = 0.666667

```

Source Code: C Program To Evaluate $\sin(x) = x - (x^3)/3! + (x^5)/5! + (x^7)/7! + ..$

[view plaincopy to clipboardprint?](#)

```

1. #include<stdio.h>
2. #include<math.h>
3.
4. double factorial(int);
5. void calc(float, float*);
6.
7. int main()
8. {
9.     int x;
10.    float radian, result = 0;
11.
12.    printf("Enter value of x in degrees\n");
13.    scanf("%d", &x);
14.
15.    radian = x * (3.14159 / 180.0); // Convert Degree To Radian
16.
17.    calc(radian, &result);
18.
19.    printf("Sin(%d) = %f\n", x, result);
20.
21.    return 0;
22. }
23.
24. void calc(float num, float *res)
25. {
26.     int count, n = 1, sign = 1;
27.
28.     for(count = 1; (n <= 10); count += 2)
29.     {
30.         *res += sign * ( pow(num, count) / factorial(count) );
31.         n += 1;
32.         sign *= -1;
33.     }
34. }
35.
36. double factorial(int num)
37. {
38.     int count;
39.     double sum = 1;

```

```
40.  
41.  for(count = 1; count <= num; count++)  
42.  {  
43.      sum *= count;  
44.  }  
45.  return(sum);  
46. }  
47. </math.h></stdio.h>
```

Output 1:

Enter value of x is degrees
0
Sin(0) = 0.000000

Output 2:

Enter value of x is degrees
30
Sin(30) = 0.500000

Output 3:

Enter value of x is degrees
45
Sin(45) = 0.707106

Output 4:

Enter value of x is degrees
60
Sin(60) = 0.866025

Output 5:

Enter value of x is degrees
90
Sin(90) = 1.000000

Check above output with the value present in below chart. If you further evaluate the values present in below chart, it matches with the output of our program:

	0°	30°	45°	60°	90°
sin	0	$\frac{1}{2}$	$\frac{\sqrt{2}}{2}$	$\frac{\sqrt{3}}{2}$	1

4. Check whether a given number is perfect or not.

[Hint: Sum of all positive divisors of a given number excluding the given number is equal to the number] Ex: $28 = 1 + 2 + 4 + 7 + 14 = 28$ is a perfect number

C For Loop: Exercise-27 with Solution

Write a c program to check whether a given number is a perfect number or not.

Pictorial Presentation:

Perfect Number

Divisor of 28 : 1, 2, 4, 7, 14, 28

Sum of 1 + 2 + 4 + 7 + 14 = 28

Sum = Original Number

28 is Perfect number

Sample Solution:

C Code:

/*Perfect number is a positive number which sum of all positive divisors excluding that number is equal to that number. For example 6 is perfect number since divisor of 6 are 1, 2 and 3. Sum of its divisor is $1 + 2 + 3 = 6$ */

```
#include <stdio.h>

void main()
{
    int n,i,sum;

    int mn,mx;

    printf("Input the number : ");

    scanf("%d",&n);

    sum = 0;

    printf("The positive divisor : ");

    for (i=1;i<n;i++)
    {
        if(n%i==0)
        {
            sum=sum+i;

            printf("%d ",i);

        }
    }

    printf("\nThe sum of the divisor is : %d",sum);

    if(sum==n)
```

```

    printf("\nSo, the number is perfect.");

else

    printf("\nSo, the number is not perfect.");

printf("\n");

}

```

Sample Output:

```

Input the number : 56
The positive divisor : 1 2 4 7 8 14 28
The sum of the divisor is : 64
So, the number is not perfect.

```

5. Find out the generic root of any number.

[Hint: Generic root is the sum of digits of a number until a single digit is obtained.]

Ex: Generic root of 456 is $4 + 5 + 6 = 15 = 1 + 5 = 6$

num = 12345;

$1 + 2 + 3 + 4 + 5 = 15$

$1 + 5 = 6$

Generic Root of number 12345 is 6.

FIND OUT GENERIC ROOT OF A NUMBER By C PROGRAM

C program to find the generic root of a number:

The below program ask the user to enter the value. After getting the value from the user it will find the generic root of a number.

```
#include <stdio.h>
int main()
{
    int num, sum, rem;
    printf("Please Enter any number = ");
    scanf("%d", &num);
    while(num >= 10)
    {
        for (sum=0; num > 0; num= num/10)
        {
            rem = num % 10;
            sum=sum + rem;
        }
        if(sum >= 10)
        {
            num = sum;
        }
        else
        {
            printf("Generic Root of Given num = %d", sum);
            break;
        }
    }
    return 0;
}
```

Output:

Please Enter any number = 123
Generic Root of Given num = 6

Version 2

```
/* C Program to Calculate Generic Root of a Number */
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int Number, Sum, Reminder;
```

```
printf("\n Please Enter any number\n");
```

```
scanf("%d", &Number);
```

```
while(Number >= 10)
```

```
{
```

```
for (Sum=0; Number > 0; Number= Number/10)
```

```
{
```

```
    Reminder = Number % 10;
```

```
    Sum=Sum + Reminder;
```

```
}
```

```
if(Sum >= 10)
```

```
{
```

```

    Number = Sum;

}

else

{

    printf("\n The Generic Root of a Given Number = %d", Sum);

    return 0;

}

}

}

```

C program for generic root

```

#include<stdio.h>
int main(){

    long int num,sum,r;
    printf("\nEnter a number:-");
    scanf("%ld",&num);

    while(num>10){
        sum=0;
        while(num){
            r=num%10;
            num=num/10;
            sum+=r;
        }
        if(sum>10)
            num=sum;
    }
}

```

```

    else
    break;
}
printf("\nSum of the digits in single digit is: %ld",sum);
return 0;
}

```

C code for calculation of generic root in one line

```

#include <stdio.h>
int main(){
    int num,x;
    printf("Enter any number: ");
    scanf("%d",&num);
    printf("Generic root: %d",(x=num%9)?x:9);
    return 0;
}

```

Sample output:

Enter any number: 731
Generic root: 2

Meaning of generic root:

It sum of digits of a number unit we don't get a single digit. For example:
Generic root of 456: $4 + 5 + 6 = 15$ since 15 is two digit numbers so $1 + 5 = 6$
So, generic root of 456 = 6

Algorithm to find the smallest and largest numbers in an array

1. Input the array elements.
2. Initialize small = large = arr[0]
3. Repeat from i = 2 to n
4. if(arr[i] > large)
5. large = arr[i]
6. if(arr[i] < small)
7. small = arr[i]
8. Print small and large.

```
#include <stdio.h>

int main()
{   int arr[100], n, i, small, large;
printf("Enter the number of elements you want to insert : ");
    scanf("%d", &n);
```

```

for (i = 0; i < n; i++)
{
    printf("Enter element %d : ", i + 1);
    scanf("%d", &arr[i]);
}
small = arr[0];
large = arr[0];
for (i = 1; i < n; i++)
{
    if (arr[i] < small)
    {
        small = arr[i];
    }
    if (arr[i] > large)
    {
        large = arr[i];
    }
}
printf("\nLargest element is : %d", large);
printf("\nSmallest element is : %d", small);
return 0;
}

```

Output

```

Enter the number of elements you want to insert : 6
Enter element 1 : 15

```

```
Enter element 2 : 2
Enter element 3 : 36
Enter element 4 : 18
Enter element 5 : 25
Enter element 6 : 30

Largest element is : 36
Smallest element is : 2
```

Here is the C program to find the largest and smallest element in a one dimensional (1-D) array.

```
1#include<stdio.h>
2
3int main()
4{
5    int a[50],i,n,large,small;
6    printf("How many elements:");
7    scanf("%d",&n);
8    printf("Enter the Array:");
9
10   for(i=0;i<n;++i)
11       scanf("%d",&a[i]);
12   large=small=a[0];
13   for(i=1;i<n;++i)
14   {
15       if(a[i]>large)
16           large=a[i];
17       if(a[i]<small)
18           small=a[i];
19   }
20   printf("The largest element is %d",large);
21   printf("\nThe smallest element is %d",small);
22
23   return 0;
24}
25
26
```

2. Write a C program to accept 'n' numbers and store all prime numbers in an array and display.

Solution:

```
#include<stdio.h>

int main()
{
    int a[10],n,i,j,c=0,prime[10],t=0;
    printf("/*How Many Numbers You Want\nTo Add in Array*/\n\nEnter Limit : ");
    scanf("%d",&n);
    printf("\nEnter %d Numbers in Array:\n\n",n);
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    for(i=0;i<n;i++)
    {
        c=0;
        for(j=2;j<a[i];j++)
        {
            if(a[i]%j==0)
            {
                c=1;
                break;
            }
        }
        if(c==0)
        {
            prime[t]=a[i];
            t++;
        }
    }
}
```

```

printf("\nPrime Numbers in Above Array:\n\n");
for(i=0;i<t;i++)
{
    printf(" %d ",prime[i]);
}
return 0;
}

```

Output:

```

/*How Many Numbers You Want
To Add in Array*/

Enter Limit : 6

Enter 6 Numbers in Array:

    12
    23
    45
    85
    449
    27

Prime Numbers in Above Array:

23  449

```

/* C program to find all prime numbers from the inputted array */

```

02  #include<stdio.h>
03  #include<conio.h>

```

```

04 void main()
05 {
06     int ar[100],i,n,j,counter;
07
08     printf("Enter
the size of the
array ");
09     scanf("%d",&n);
10     printf("\n
Now enter
the elements
of the
array");
11     for(i=0;i<n;i++)
12     {
13         scanf("%d",&ar[i]);
14     }
15
16     printf("
Array is -
");
17     for(i=0;i<n;i++)
18     {
19         printf("\t
%d",ar[i]);
20     }
21
22     printf("\n
All the
prime
numbers in
the array are
-");
23     for(i=0;i<n;i++)
24     {
25         counter=0;
26         for(j=2;j<ar[i];j++)
27         {
28             if(ar[i]%j==0)

```

```

29         {
30             counter=1;
31             break;
32         }
33     }
34     if(counter==0)
35     {
36         printf("\t
%d",ar[i]);
37     }
38 }
39 getch();
40 }

```

Output –

Enter the size of the array – 5

Now enter the elements of the array – 23 98 45 101 6

Array is – 23 98 45 101 6

All the prime numbers in the array are – 23 101

How to Use Bubble Sort in C Programming?

[View More](#)

There might come situations when you would want to order data in a specific order. That's when a sorting algorithm will come in handy. Bubble sort in C is a straightforward [sorting algorithm](#) that checks and swaps elements if they are not in the intended order. It compares two adjacent elements to find which one is greater or lesser and switches them based on the given condition until the final place of the element is found. In this article, you will learn about bubble sort and how to write a [C program](#) for bubble sort implementation using different ways.

Working of Bubble Sort

Bubble sort is a data sorting algorithm that works by randomly copying elements from the first array into a smaller second array, and then reversing the order of these arrays. After

this process has been repeated multiple times, the sorted data will be located in the middle of the larger array.

The basic idea behind the bubble sort is to compare the elements of an array one by one until they are sorted in ascending order, which is called bubble bursting. When an element needs to be moved, instead of moving the entire array, only the element affected by the change is moved. This technique conserves memory and keeps overall execution speed high because there are fewer updates than with other sorting algorithms.

1. First Iteration (Compare and Swap)

Bubble Sort is a sorting algorithm that works by first sorting the items into two piles, and then swapping the items in each pile until they are sorted in reverse order. This process is known as the First Iteration of Bubble Sort.

For example, we need to sort these elements -5, 72, 0, 33, - 9, then the sequence will work in this way.

- Starting with the first index, the first and second components should be juxtaposed.
- The first and second elements are switched if the first one is bigger.
- Compare the second and third items right now. If they are not in the proper sequence, swap them.
- The method described above continues until the last component.

Bubble sort first iteration in C program

```

#include<stdio.h>

int main() {

double Array[5]; // array to be sorted

double temp; // temporary variable to hold the current element of the array

void bubbleSort(Array); // function that performs bubble sort on a given array

while (true) {

temp=Array[0]; /* swap two elements */

Arrays[1]=Arrays[2]; /* update second position */

Arrows ++; /* make next move */

}/*endwhile*/ // loop body }// end of Bubble Sort First Iteration in C programming
language

```

2. *Remaining Iteration*

The Bubble Sort is an efficient sorting algorithm that works in $O(n \log n)$ time, where n is the number of items to be sorted. The first iteration of the Bubble Sort sorts the input item at index 0 into ascending order, and then repeats this process until all the inputs have been sorted. So, after performing one iteration of the bubble sort on an input dataset containing five items, there would be four remaining iterations left to perform.

The Bubble Sort Algorithm in C

The basic bubble sort algorithm can be explained as follows:

```
bubbleSort(array)

for i <- 1 to indexOfLastUnsortedElement-1

    if leftElement > rightElement

        swap leftElement and rightElement

end bubbleSort
```

This [algorithm](#) does the swapping of elements to get the final output in the desired order. For instance, if you pass an array consisting of the elements: (6, 3, 8, 2, 5, 7), the final array after the bubble sort implementation will be: (2, 3, 5, 6, 7, 8).

Read more: [Array in C](#)

Bubble Sort Data Structure Example in C - Program to arrange elements in Ascending, Descending Order

Bubble Sort is a simple method to sort list, in this sorting technique we compare adjacent elements and swap if they are in a wrong order. This process repeats until no more swaps are needed.

C program for Data Structure Bubble Sort Example - In this program we will read N number of elements in a One Dimensional Array and arrange all elements in **Ascending and Descending Order** using **Data Structure Bubble Sort technique**.

Data Structure - Bubble Sort Example using C program

```
/*Bubble Sort - C program to sort an Array
in Ascending and Descending Order.*/

#include <stdio.h>

#define MAX 100

int main()
{
    int arr[MAX],limit;
    int i,j,temp;

    printf("Enter total number of elements: ");
    scanf("%d",&limit);

    /*Read array*/
    printf("Enter array elements: \n");
    for(i=0; i<limit; i++)
    {
        printf("Enter element %3d: ",i+1);
        scanf("%d",&arr[i]);
    }

    /*sort elements in Ascending Order*/
    for(i=0; i<(limit-1); i++)
    {
        for(j=0; j<(limit-i-1); j++)
        {
            if(arr[j]>arr[j+1])
            {
                temp=arr[j];
                arr[j]=arr[j+1];
                arr[j+1]=temp;
            }
        }
    }
}
```

```

}

printf("Array elements in Ascending Order:\n");
for(i=0; i<limit; i++)
    printf("%d ",arr[i]);

printf("\n");

/*sort elements in Descending Order*/
for(i=0; i<(limit-1); i++)
{
    for(j=0; j<(limit-i-1); j++)
    {
        if(arr[j]<arr[j+1])
        {
            temp=arr[j];
            arr[j]=arr[j+1];
            arr[j+1]=temp;
        }
    }
}

printf("Array elements in Descending Order:\n");
for(i=0; i<limit; i++)
    printf("%d ",arr[i]);

printf("\n");

return 0;
}

```

Output

```

Enter total number of elements: 10
Enter array elements:
Enter element 1: 12
Enter element 2: 34
Enter element 3: 43

```

```
Enter element 4: 32
Enter element 5: 21
Enter element 6: 1
Enter element 7: 11
Enter element 8: 2
Enter element 9: 3
Enter element10: 100
Array elements in Ascending Order:
1 2 3 11 12 21 32 34 43 100
Array elements in Descending Order:
100 43 34 32 21 12 11 3 2 1
```

C program to arrange numbers in ascending & descending order

Source Code:

```
#include<stdio.h>

int main()
{
    int i,j,a,n,ch;
    printf("Enter the Value of N \n");
    scanf("%d",&n);
    int number[n];
    printf("Enter the Numbers \n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&number[i]);
    }
    while(1)
    {
        printf("Press 1. For ASCENDING ORDER\n");
        printf("Press 2. For DESCENDING ORDER\n");
        printf("Enter Your Choice:\n");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1: for(i=0;i<n;i++)
```

```

{
for(j=i+1;j<n;j++)
{
if(number[i]>number[j])
{
a=number[i];
number[i]=number[j];
number[j]=a;
}
}
}
printf("\nThe numbers arranged in ascending order are given below\n");
for(i=0;i<n;i++)
{
printf("%d\n",number[i]);
}
break;
case 2:
for(i=0;i<n;i++)
{
for(j=i+1;j<n;j++)
{
if(number[i]<number[j])
{
a=number[i];
number[i]=number[j];
number[j]=a;
}
}
}
}
printf("\nThe numbers arranged in descending order are given below\n");
for(i=0;i<n;i++)
{
printf("%d\n",number[i]);
}
break;
default:printf("\n INVALID CHOICE \n");
break;
}

```

```
}  
return 0;  
}
```

Output:

Enter the value of N: 5

Enter the numbers

15

65

80

10

101

Press 1. For ASCENDING ORDER

Press 2. For DESCENDING ORDER

Enter Your Choice:

1

The numbers arranged in ascending order are given below

10

15

65

80

101

Press 1. For ASCENDING ORDER

Press 2. For DESCENDING ORDER

Enter Your Choice:

2

The numbers arranged in descending order are given below

101

80

65

15

10

Press 1. For ASCENDING ORDER

Press 2. For DESCENDING ORDER

Enter Your Choice:

4

INVALID CHOICE

Press 1. For ASCENDING ORDER

Press 2. For DESCENDING ORDER

Enter Your Choice:

How Does the C Program for Bubble Sort Work?

As mentioned, the [C program](#) for bubble sort works by comparing and swapping adjacent elements in an array. Let's understand this in a step-by-step method:

Suppose we want to sort an array, let's name it arr, with n elements in ascending order; this is how the bubble sort algorithm will work.

1. Starts from the first index: arr[0] and compares the first and second element: arr[0] and arr[1]
2. If arr[0] is greater than arr[1], they are swapped
3. Similarly, if arr[1] is greater than arr[2], they are swapped

4. The above process continues until the last element `arr[n-1]`

All four steps are repeated for each iteration. Upon completing each iteration, the largest unsorted element is moved to the end of the array. Finally, the program ends when no elements require swapping, giving us the array in ascending order. Now that we know the working of the bubble sort let's implement it in C programming using different methods.

4. C Program Insert an element into a 1D array by getting an element and the position from the user.

C program to insert an element in array

[July 11, 2015PankajC programmingArray, C, Program](#)

Write a C program to insert element in array at specified position. C program to insert element in array at given position. The program should also print an error message if the insert position is invalid. Logic to insert an element in array at given position in C program.

Example

Input

Input array elements: 10, 20, 30, 40, 50

Input element to insert: 25

Input position where to insert: 3

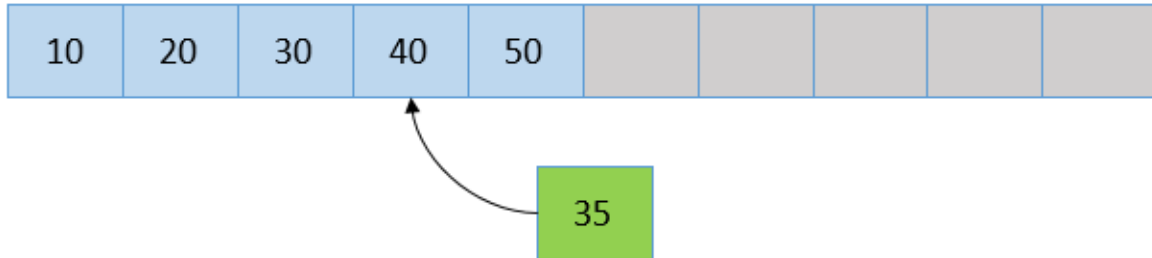
Output

Elements of array are: 10, 20, 25, 30, 40, 50

Logic to insert element in array

Step by step descriptive logic to insert element in array.

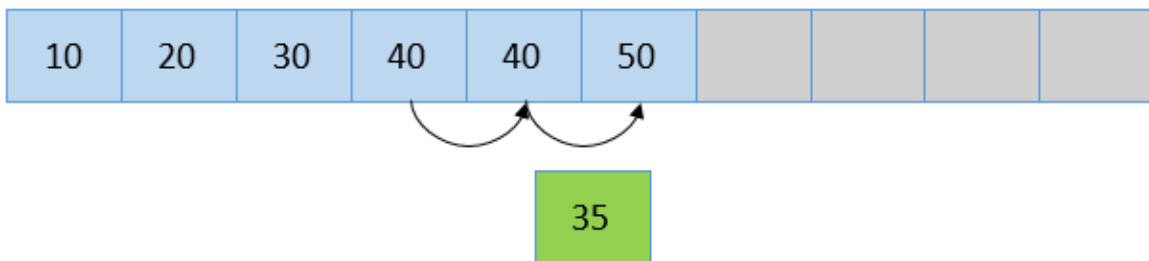
1. [Input size and elements in array](#). Store it in some variable say `size` and `arr`.
2. Input new element and position to insert in array. Store it in some variable say `num` and `pos`.



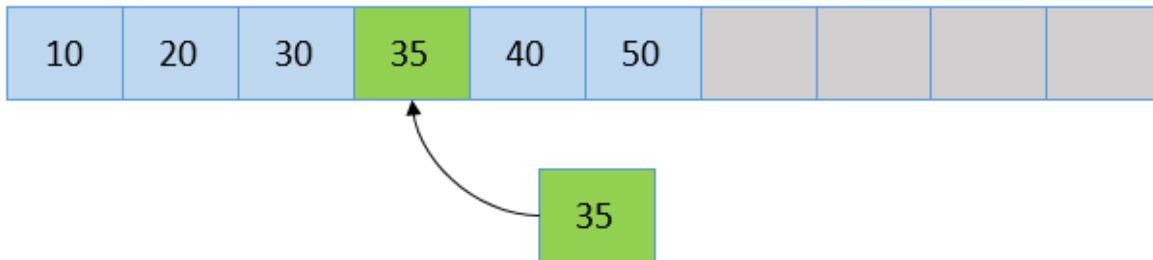
3. To insert new element in array, shift elements from the given insert position to one position right. Hence, run a loop in descending order from `size` to `pos` to insert. The loop structure should look like `for(i=size; i>=pos; i--)`.

Inside the loop copy previous element to current element by `arr[i] = arr[i - 1];`.

Read more - [C program to copy array element](#)



4. Finally, after performing shift operation. Copy the new element at its specified position i.e. `arr[pos - 1] = num;`



Program to insert element in array

```
1 #include<stdio.h>
2
3 int main() {
4     int arr[30], element, num, i, location;
5
6     printf("\nEnter no of elements :");
7     scanf("%d", &num);
8
9     for (i = 0; i < num; i++) {
10         scanf("%d", &arr[i]);
11     }
12
13     printf("\nEnter the element to be inserted :");
14     scanf("%d", &element);
15
16     printf("\nEnter the location");
17     scanf("%d", &location);
18
19     //Create space at the specified location
20     for (i = num; i >= location; i--) {
21         arr[i] = arr[i - 1];
```

```

22 }
23
24 num++;
25 arr[location - 1] = element;
26
27 //Print out the result of insertion
28 for (i = 0; i < num; i++)
29     printf("n %d", arr[i]);
30
31 return (0);
32 }

```

Output of the Program :

```

1 Enter no of elements : 5
2 1 2 3 4 5
3 Enter the element to be inserted : 6
4 Enter the location : 2
5 1 6 2 3 4 5

```

Write a Program in c to insert an element at given position in an array

```

/* C Program to to insert an element at given position in an array */

# include < stdio.h >

int main( )
{
    int a[20], i, n, ele, pos ;
    printf(" Enter the Numbers of elements: ") ;
    scanf("%d",&n) ;
    printf("\n Enter the elements of array : \n") ;
    for ( i = 1 ; i <= n ; i++)
        scanf("%d",&a[i]) ;
    printf("\n Array enter by user are :\n") ;
    for ( i = 1 ; i <= n ; i++)
        printf("%d\t",a[i]) ;

    printf("\n Enter the position you want to enter :") ;
    scanf("%d",&pos) ;

```

```

printf("\n Enter the element you want to enter :");
scanf("%d",&ele);

for ( i = 1 ; i <= n ; i++)
{
    if ( i > pos )
    a[i] = a[i-1] ;
    else
    {
        if ( i == pos )
        a[i] = ele ;
        else
        a[i] = a[i] ;
    }
}

printf("\n Array After Inserting element :\n");
for ( i = 1 ; i <= n ; i++)
printf("%d\t",a[i]);
return ( 0 ) ;
}

```

Output of Program:

```

Enter the Number of position: 5
Enter the elements of array :
12
32
22
41
23

Array enter by user are :
12      32      22      41      23
Enter the position you want to enter :3
Enter the element you want to enter :55
Array After Inserting element :
12      32      55      22      41      23

```

5. C program to Search the position of the number that is entered by the user and delete that number from the array and display the resultant array elements.

Remove an element from an array in C

In this topic, we will learn how to delete or remove a particular element from an array in the C programming language. An array is the collection of the same data type elements or items stored in a contiguous memory block. In C programming, an array is derived data that stores primitive data type values like int, char, float, etc.

To delete a specific element from an array, a user must define the position from which the array's element should be removed. The deletion of the element does not affect the size of an array. Furthermore, we should also check whether the deletion is possible or not in an array.

For example, suppose an array contains seven elements, `arr[] = {10, 25, 14, 8, 12, 15, 5};` and the user want to delete element 8. So, first, the user must define the position of the 8th element, which is the 4th, and then check whether the deletion is possible or not. The position of the particular element should not be more than the total elements of an array. Here, we have 7 elements in an array, and the user wants to delete the 8th position element, which is impossible.

Steps to remove an element from an array

Following is the steps to remove a particular element from an array in C programming

Step 1: Input the size of the array `arr[]` using `num`, and then declare the `pos` variable to define the position, and `i` represent the counter value.

Step 2: Use a loop to insert the elements in an array until (`i < num`) is satisfied.

Step 3: Now, input the position of the particular element that the user or programmer wants to delete from an array.

Step 4: Compare the position of an element (`pos`) from the total no. of elements (`num+1`). If the `pos` is greater than the `num+1`, the deletion of the element is not possible and jump to step 7.

Step 5: Else removes the particular element and shift the rest elements' position to the left side in an array.

Step 6: Display the resultant array after deletion or removal of the element from an array.

Step 7: Terminate or exit from the program.

Example 1: Program to remove an element from an array using for loop

Let's create a program to delete an element from an array using for loop in the C programming language.

```
1. /* program to remove the specific elements from an array in C. */
2. #include <stdio.h>
3. #include <conio.h>
4.
5. int main ()
6. {
7.     // declaration of the int type variable
8.     int arr[50];
9.     int pos, i, num; // declare int type variable
10.    printf (" \n Enter the number of elements in an array: \n ");
11.    scanf (" %d", &num);
12.
13.    printf (" \n Enter %d elements in array: \n ", num);
14.
15.    // use for loop to insert elements one by one in array
16.    for (i = 0; i < num; i++ )
17.    { printf (" arr[%d] = ", i);
18.        scanf (" %d", &arr[i]);
19.    }
20.
21.    // enter the position of the element to be deleted
```

```

22.  printf( " Define the position of the array element where you want to delete: \n ");

23.  scanf( " %d", &pos);
24.
25.  // check whether the deletion is possible or not
26.  if (pos >= num+1)
27.  {
28.      printf( " \n Deletion is not possible in the array.");
29.  }
30.  else
31.  {
32.      // use for loop to delete the element and update the index
33.      for (i = pos - 1; i < num -1; i++)
34.      {
35.          arr[i] = arr[i+1]; // assign arr[i+1] to arr[i]
36.      }
37.
38.      printf( " \n The resultant array is: \n");
39.
40.      // display the final array
41.      for (i = 0; i< num - 1; i++)
42.      {
43.          printf( " arr[%d] = ", i);
44.          printf( " %d \n", arr[i]);
45.      }
46.  }
47.  return 0;
48. }

```

Output:

```
Enter the number of elements in an array:
8

Enter 8 elements in array:
arr[0] = 3
arr[1] = 6
arr[2] = 2
arr[3] = 15
arr[4] = 10
arr[5] = 5
arr[6] = 8
arr[7] = 12
Define the position of the array element where you want to delete:
5

The resultant array is:
arr[0] = 3
arr[1] = 6
arr[2] = 2
arr[3] = 15
arr[4] = 5
arr[5] = 8
arr[6] = 12
```

In the above program, we input 8 elements for an array `arr[]` from the user. After that, the user enters the position of the particular element is 5. And then, we checked the defined position by the If statement (`if (pos > num + 1)`); Here, 5 is less than the (`num+1`). So the condition is false, and it executes the else block to remove the 5th position of the element is 10 using for loop and then prints the resultant array.

Example 2: Program to remove the specific character from an array using for loop

Let's consider an example to delete the particular position of the defined character from an array using for loop in the C programming language.

```

1.  /* program to remove the specific elements from an array in C. */
2.  #include <stdio.h>
3.  #include <conio.h>
4.
5.  int main ()
6.  {
7.      // declaration of the int type variable
8.      int arr[50];
9.      int pos, i, num; // declare int type variable
10.     printf ( " \n Enter the number of elements in an array: \n ");
11.     scanf ( " %d", &num);
12.
13.     printf ( " \n Enter %d elements in array: \n ", num);
14.
15.     // use for loop to insert elements one by one in array
16.     for (i = 0; i < num; i++ )
17.     { printf ( " arr[%d] = ", i);
18.         scanf ( " %c", &arr[i]);
19.     }
20.
21.     // enter the position of the element to be deleted
22.     printf( " Define the position of the array element where you want to delete: \n ");
23.     scanf ( " %d", &pos);
24.
25.     // check whether the deletion is possible or not
26.     if (pos >= num+1)

```

```
27.  {
28.    printf (" \n Deletion is not possible in the array.");
29.  }
30.  else
31.  {
32.    // use for loop to delete the element and update the index
33.    for (i = pos - 1; i < num -1; i++)
34.    {
35.        arr[i] = arr[i+1]; // assign arr[i+1] to arr[i]
36.    }
37.
38.    printf (" \n The resultant array is: \n");
39.
40.    // display the final array
41.    for (i = 0; i< num - 1; i++)
42.    {
43.        printf (" arr[%d] = ", i);
44.        printf (" %c \n", arr[i]);
45.    }
46.  }
47.  return 0;
48. }
```

Output:

Enter the number of elements in an array:

8

Enter 8 elements in array:

arr[0] = a

arr[1] = b

arr[2] = f

arr[3] = h

```
arr[4] = e
arr[5] = k
arr[6] = w
arr[7] = p
Define the position of the array element where you want to delete:
4
```

The resultant array is:

```
arr[0] = a
arr[1] = b
arr[2] = f
arr[3] = e
arr[4] = k
arr[5] = w
arr[6] = p
```

Remove element from array C program

```
#include <stdio.h>
int main()
{
    int array[100], position, c, n;

    printf("Enter number of elements in array\n");
    scanf("%d", &n);

    printf("Enter %d elements\n", n);

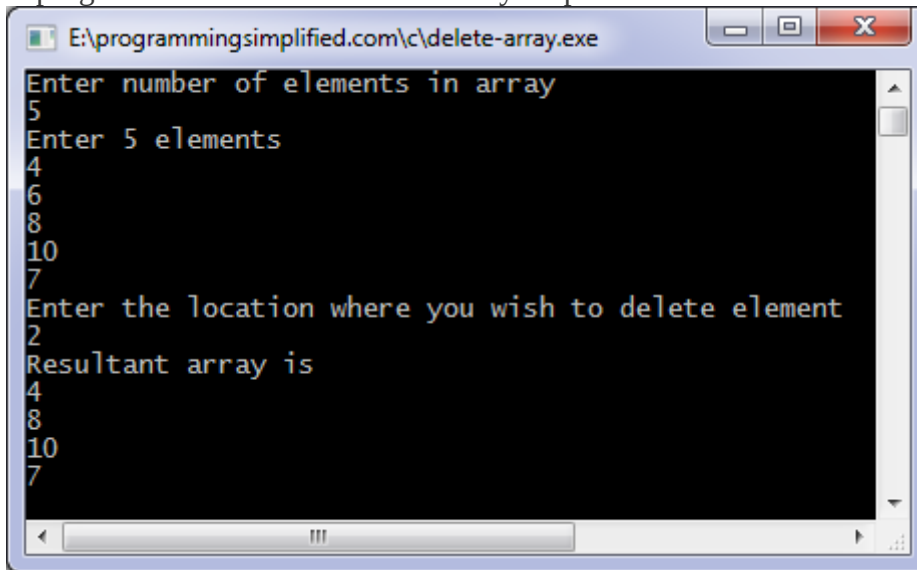
    for (c = 0; c < n; c++)
        scanf("%d", &array[c]);

    printf("Enter the location where you wish to delete element\n");
    scanf("%d", &position);

    if (position >= n+1)
        printf("Deletion not possible.\n");
    else
    {
        for (c = position - 1; c < n - 1; c++)
            array[c] = array[c+1];
```

```
printf("Resultant array:\n");  
  
for (c = 0; c < n - 1; c++)  
    printf("%d\n", array[c]);  
}  
  
return 0;  
}
```

C program to delete element from array output:



The screenshot shows a Windows command prompt window titled "E:\programmingsimplified.com\c\delete-array.exe". The program prompts the user to "Enter number of elements in array", where the input "5" is shown. It then prompts "Enter 5 elements", with the inputs "4", "6", "8", "10", and "7" shown on separate lines. Next, it prompts "Enter the location where you wish to delete element", with the input "2" shown. Finally, it displays "Resultant array is" followed by the elements "4", "8", "10", and "7" on separate lines. The window has standard Windows controls (minimize, maximize, close) in the title bar.

```
E:\programmingsimplified.com\c\delete-array.exe  
Enter number of elements in array  
5  
Enter 5 elements  
4  
6  
8  
10  
7  
Enter the location where you wish to delete element  
2  
Resultant array is  
4  
8  
10  
7
```

Write C programs to do the following:

1. Find whether a given matrix is symmetric or not. [Hint: $A = A^T$]

What is Symmetric Matrix?

Symmetric matrix is a square matrix which is equal to its transpose. A symmetric matrix is always a square matrix. Symmetric matrix **A** is defined as - **$A = A^T$**

$$\begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 5 \\ 3 & 5 & 8 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 5 \\ 3 & 5 & 8 \end{bmatrix}^T$$

Symmetric matrix

Logic to check symmetric matrix

To check whether a matrix **A** is symmetric or not we need to check whether **$A = A^T$** or not. Below is the step by step descriptive logic to check symmetric matrix.

1. Input elements in matrix **A**.
2. Find transpose of matrix **A**, store it in some variable say **B**.

3. Check if matrix A is equal to its transpose A^T then it is symmetric matrix otherwise not. Means check if $A_{ij} = A^T_{ij}$ (Where $1 \leq i \leq m$ and $1 \leq j \leq n$) then the matrix is symmetric.

```
/* C Program to check Matrix is a Symmetric Matrix or Not */
```

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int i, j, rows, columns, a[10][10], b[10][10], Count = 1;
```

```
    printf("\n Please Enter Number of rows and columns : ");
```

```
    scanf("%d %d", &i, &j);
```

```
    printf("\n Please Enter the Matrix Elements \n");
```

```
    for(rows = 0; rows < i; rows++)
```

```
    {
```

```
        for(columns = 0; columns < j; columns++)
```

```
        {
```

```
            scanf("%d", &a[rows][columns]);
```

```
        }
```

```
    }
```

```
    //Transpose of matrix
```

```
    for(rows = 0; rows < i; rows++)
```

```
    {
```

```
        for(columns = 0; columns < j; columns++)
```

```
        {
```

```
            b[columns][rows] = a[rows][columns];
```

```
        }
```

```
    }
```

```
    for(rows = 0; rows < i; rows++)
```

```

{
    for(columns = 0; columns < j; columns++)
    {
        if(a[rows][columns] != b[rows][columns])
        {
            Count++;
            break;
        }
    }
}
if(Count == 1)
{
    printf("\n The Matrix that you entered is a Symmetric Matrix ");
}
else
{
    printf("\n The Matrix that you entered is Not a Symmetric Matrix ");
}
return 0;
}

```

Output

Enter elements in matrix of size 3x3:

1 2 3

2 4 5

3 5 8

The given matrix is Symmetric matrix:

1 2 3

2 4 5

3 5 8

- Find the trace and norm of a given square matrix.
Hint: Trace= sum of principal diagonal elements
Norm= SQRT (sum of squares of the individual elements of an array)]

What are Normal and Trace of a Matrix?

The square root of the sum of the squares of each element of the matrix is said to be **Normal** of a matrix.

The sum of the diagonal elements of a matrix is called **Trace**. Let's consider the following matrix A.

$$\begin{bmatrix} 1 & 3 & 5 \\ 4 & 2 & 6 \\ 5 & 9 & 2 \end{bmatrix}$$

Here, the elements which are highlighted are diagonal elements of the matrix. If we sum them, we will find trace of the matrix as follows :-

$$\text{Trace of A} = 1 + 2 + 2 = 5$$

To find the normal, we will square each element and find sum of those squared elements. After that, we will calculate square root of the sum calculated. The resulting number will be Normal of our matrix. Let's calculate Normal for A.

$$\text{Normal} = (\sqrt{1^2 + 3^2 + 5^2 + 4^2 + 2^2 + 6^2 + 5^2 + 9^2 + 2^2})$$

$$\text{Normal} = (\sqrt{195})$$

Normal = 13.96

Few important points to remember:

- **Normal** and **Trace** are only defined for a square matrix.
- **Square Matrix:** Matrix in which, the **number of rows = number of columns**.
- **Diagonal Element:** An element having same indices for row and column.

Algorithm for Normal and Trace of a Square Matrix Program in C

To find Trace of a matrix, the algorithm is as below:

1. Let the matrix be **A[m][n]**
2. trace = 0
3. for i = 0 to m
4. for j = 0 to n
5. if i == j
6. trace += A[i][j]

The algorithm for finding Normal of a matrix is as follows:

1. Let the matrix be **A[m][n]**

2. sum = 0, normal = 0

```
3. for i = 0 to m
```

```
4. for j = 0 to n
```

```
5. A[i][j] *= A[i][j], sum += A[i][j]
```

```
6. normal = sqrt(sum)
```

C Program to find Normal and Trace of Square Matrix

So let's see the complete program now. We are using the **math.h** header library in this program.

In some latest versions of C compilers (gcc), you have to **explicitly specify** that you want to use **math.h** header in the program, when you compile it.

You can do so by adding the **-lm** flag at the end of the gcc command. For example, **gcc program.c -o program -lm**

So when you run this code in your local setup, remember this.

Problem Description

This C Program find the trace & normal of a given matrix. Here trace of the matrix is the sum of the elements of the main diagonal i.e the diagonal from the upper left to the lower right of a matrix. Normal of the matrix is the square root of the sum of all the elements.

Problem Solution

1. Create a matrix and define all its elements.
2. To evaluate normal of the matrix, take sum of all the elements of the array and calculate the square root of it.
3. To evaluate trace of the matrix, take sum of the main diagonal elements.

Program/Source Code

Here is source code of the C program to find the trace & normal of a given matrix. The program is successfully compiled and tested using Turbo C compiler in windows environment. The program output is also shown below.

```
1.  /*
2.   * C program to find the trace and normal of a matrix
3.   *
4.   * Trace is defined as the sum of main diagonal elements and
5.   * Normal is defined as square root of the sum of all the elements
6.   */
7.  #include <stdio.h>
8.  #include <math.h>
9.  void main ()
10. {
11.     static int array[10][10];
12.     int i, j, m, n, sum = 0, sum1 = 0, a = 0, normal;
13.     printf("Enter the order of the matrix\n");
14.     scanf("%d %d", &m, &n);
15.     printf("Enter the n coefficients of the matrix \n");
16.     for (i = 0; i < m; ++i)
17.     {
18.         for (j = 0; j < n; ++j)
19.         {
20.             scanf("%d", &array[i][j]);
21.             a = array[i][j] * array[i][j];
22.             sum1 = sum1 + a;
23.         }
24.     }
25.     normal = sqrt(sum1);
26.     printf("The normal of the given matrix is = %d\n", normal);
```

```

27.     for (i = 0; i < m; ++i)
28.     {
29.         sum = sum + array[i][i];
30.     }
31.     printf("Trace of the matrix is = %d\n", sum);
32. }

```

Program Explanation

1. Declare a matrix and define all its elements.
2. Declare variables for storing the normal and trace of the matrix.
3. Find the sum of all the elements of the matrix using nested for loop.
4. Evaluate normal by passing the above calculated sum to sqrt() function.
5. Take sum of all the main diagonal elements of the array to calculate trace.
6. Print trace and normal.

3. Perform matrix multiplication.

Introduction to Matrix Multiplication in C

An ordered rectangular array of numbers or functions is known as a matrix. A rectangular array of $m \times n$ numbers in the form of n vertical lines called columns and m horizontal lines called rows is called a matrix of order m by n .

Two matrices can be multiplied only if the number of columns in the first matrix equals the number of rows in the second matrix. The product of the two matrices will have the order of the number of rows in the first row and the number of columns in the second matrix.

The product of two matrices, A and B , is the sum of the products across some row of A with the corresponding entries down some column of B . In simple words, The dot product of the first row of the first matrix and the first column of the second matrix will result in the first element of the product matrix.

This can be understood easily by the following illustration.

$$A = \begin{bmatrix} 1 & -1 & 2 \\ 0 & 3 & 4 \end{bmatrix} \quad B = \begin{bmatrix} 2 & 7 \\ -1 & 1 \\ 5 & -4 \end{bmatrix}$$

Step 1

$$\begin{bmatrix} 1 & -1 & 2 \\ 0 & 3 & 4 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \\ 5 \end{bmatrix} = \begin{bmatrix} (1)(2)+(-1)(-1)+(2)(5) & _ & _ \\ _ & _ & _ \end{bmatrix}$$

Step 2

$$\begin{bmatrix} 1 & -1 & 2 \\ 0 & 3 & 4 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \\ 5 \end{bmatrix} = \begin{bmatrix} 13 & 1(7)+(-1)(1)+2(-4) & _ \\ _ & _ & _ \end{bmatrix}$$

Step 3

$$\begin{bmatrix} 1 & -1 & 2 \\ 0 & 3 & 4 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \\ 5 \end{bmatrix} = \begin{bmatrix} 13 & -2 & _ \\ 0(2)+3(-1)+4(5) & _ & _ \end{bmatrix}$$

Step 4

$$\begin{bmatrix} 1 & -1 & 2 \\ 0 & 3 & 4 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \\ 5 \end{bmatrix} = \begin{bmatrix} 13 & -2 & _ \\ 17 & 0(7)+3(1)+4(4) & _ \end{bmatrix}$$

Final solution

$$AB = \begin{bmatrix} 13 & -2 \\ 17 & -13 \end{bmatrix}$$

SCALER
Topics

Algorithm of C Programming Matrix Multiplication

1. Start.
2. Enter the value of m and n (or) order of the first matrix.
3. Enter the value of p and q (or) order of the second matrix.
4. Create a matrix of size a[m][n] and b[p][q].
5. Enter the element of matrices row-wise using loops.
6. If the number of columns of the first matrix is not equal to the number of rows of the second matrix, print matrix multiplication is not possible and exit. If not, proceed to the next step.
7. Create a third matrix, c of size m x q, to store the product.
8. Set a loop from i=0 to i=m.
9. Set an inner loop for the above loop from j=0 to j=q.
10. Initialise the value of the element (i, j) of the new matrix to 0.
11. Set an inner loop inside the above loop from k=0 to k=p.
12. Using the add and assign operator (+=) store the value of a[i][k] * b[k][j] in the third matrix, c[i][j].
13. Print the third matrix.
14. Stop.

Multiplication of Two Matrices in C

There are mainly two types of matrices:

- Square matrix
- Rectangle matrix

Let's first start with square matrix multiplication.

Multiplication of Square Matrices

A matrix in which the number of rows equals the number of columns is called a square matrix. Multiplication of two square matrices of the same order is possible only when both the matrices have the same order.

To calculate the product of the two matrices, we multiply the corresponding elements and add the products together. It can be understood even better with the illustration which was mentioned under the heading introduction to matrix multiplication in C.

Given below is the code to find the product of two square matrices.

```
#include<stdio.h>
int main() {
    int a[10][10], b[10][10], c[10][10], n, i, j, k;

    printf("Enter the value of N (N <= 10): ");
    scanf("%d", & n);
    printf("Enter the elements of Matrix-A: \n");

    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", & a[i][j]);
        }
    }

    printf("Enter the elements of Matrix-B: \n");
```

```

for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        scanf("%d", & b[i][j]);
    }
}

for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        c[i][j] = 0;
        for (k = 0; k < n; k++) {
            c[i][j] += a[i][k] * b[k][j];
        }
    }
}

printf("The product of the two matrices is: \n");
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        printf("%d\t", c[i][j]);
    }
    printf("\n");
}
return 0;
}

```

Notice that in the above program, we asked the user to enter the order of the matrix only once. It is because the multiplication of two square matrices of a different order is not possible.

output

```

Enter the value of N (N <= 10): 2
Enter the elements of Matrix-A:
2 2
2 2
Enter the elements of Matrix-B:
2 2
2 2
Product of the two matrices is :

```

```
8 8
8 8
```

Multiplication of Rectangular Matrices

A matrix in which the number of rows is not equal to the number of columns is called a rectangular matrix.

The criteria for the compatibility of multiplication of two rectangular matrices remains the same as the criteria for multiplication of two square matrices, i.e, the number of columns in the first matrix should be equal to the number of rows in the second matrix.

Given below is a code to find the product of two rectangular matrices.

```
#include<stdio.h>
int main() {
    int m, n, p, q, i, j, k;
    int a[10][10], b[10][10], res[10][10];

    printf("Enter the order of first matrix\n");
    scanf("%d%d", &m, &n);
    printf("Enter the order of second matrix\n");
    scanf("%d%d", &p, &q);

    if (n != p) {
        printf("Matrix is incompatible for multiplication\n");
    } else {
        printf("Enter the elements of Matrix-A:\n");
        for (i = 0; i < m; i++) {
            for (j = 0; j < n; j++) {
                scanf("%d", &a[i][j]);
            }
        }

        printf("Enter the elements of Matrix-B:\n");
        for (i = 0; i < p; i++) {
            for (j = 0; j < q; j++) {
                scanf("%d", &b[i][j]);
            }
        }
    }
}
```

```

for (i = 0; i < m; i++) {
    for (j = 0; j < q; j++) {
        res[i][j] = 0;
        for (k = 0; k < p; k++) {
            res[i][j] += a[i][k] * b[k][j];
        }
    }
}

printf("The product of the two matrices is:-\n");

for (i = 0; i < m; i++) {
    for (j = 0; j < q; j++) {
        printf("%d\t", res[i][j]);
    }
    printf("\n");
}

return 0;
}

```

When the number of columns in the first matrix is equal to the number of rows in the second matrix.

output

```

Enter the order of first matrix
2 3
Enter the order of second matrix
3 2
Enter the elements of Matrix-A:
1 2
1 2
1 2
Enter the elements of Matrix-B:
1 2 3
1 2 3

```

Product of the two matrices is
The product of the two matrices is:-

9 7
9 11

Let us try running the program with the number of columns in the first matrix not equal to the number of rows in the second matrix.

output

```
Enter the order of first matrix
1 2
Enter the order of second matrix
1 2
Matrix is incompatible for multiplication
```

1. `#include<stdio.h>`
2. `#include<stdlib.h>`
3. `int main(){`
4. `int a[10][10],b[10][10],mul[10][10],r,c,i,j,k;`
5. `system("cls");`
6. `printf("enter the number of row=");`
7. `scanf("%d",&r);`
8. `printf("enter the number of column=");`
9. `scanf("%d",&c);`
10. `printf("enter the first matrix element=\n");`
11. `for(i=0;i<r;i++)`
12. `{`
13. `for(j=0;j<c;j++)`
14. `{`
15. `scanf("%d",&a[i][j]);`
16. `}`
17. `}`
18. `printf("enter the second matrix element=\n");`

```
19. for(i=0;i<r;i++)
20. {
21.     for(j=0;j<c;j++)
22.     {
23.         scanf("%d",&b[i][j]);
24.     }
25. }
26.
27. printf("multiply of the matrix=\n");
28. for(i=0;i<r;i++)
29. {
30.     for(j=0;j<c;j++)
31.     {
32.         mul[i][j]=0;
33.         for(k=0;k<c;k++)
34.         {
35.             mul[i][j]+=a[i][k]*b[k][j];
36.         }
37.     }
38. }
39. //for printing result
40. for(i=0;i<r;i++)
41. {
42.     for(j=0;j<c;j++)
43.     {
44.         printf("%d\t",mul[i][j]);
45.     }
46.     printf("\n");
47. }
```

```
48. return 0;
49. }
```

Output:

```
enter the number of row=3
enter the number of column=3
enter the first matrix element=
1 1 1
2 2 2
3 3 3
enter the second matrix element=
1 1 1
2 2 2
3 3 3
multiply of the matrix=
6 6 6
12 12 12
18 18 18
```

4.To interchange the primary and secondary diagonal elements in the given Matrix.

Program to Interchange Diagonals of Matrix

- Difficulty Level : [Medium](#)
- Last Updated : 14 Jul, 2022

☐ Read

☐ Discuss

Given a square matrix of order $n \times n$, you have to interchange the elements of both diagonals.

Examples :

Input : matrix[][] = {1, 2, 3,
4, 5, 6,
7, 8, 9}

Output : matrix[][] = {3, 2, 1,
4, 5, 6,
9, 8, 7}

Input : matrix[][] = {4, 2, 3, 1,
5, 7, 6, 8,
9, 11, 10, 12,
16, 14, 15, 13}

Output : matrix[][] = {1, 2, 3, 4,
5, 6, 7, 8,
9, 10, 11, 12,
11, 14, 15, 16}

Explanation: Idea behind interchanging diagonals of a square matrix is simple. Iterate from 0 to n-1 and for each iteration you have to swap $a[i][i]$ and $a[i][n-i-1]$.

C Program to Interchange Diagonals of a Matrix

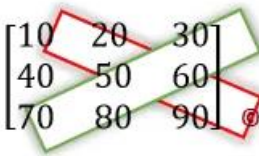
How to write a C Program to Interchange Diagonals of a Matrix with an example?.

3 * 3 Matrix

Interchange Diagonals

Final Matrix

$$\begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{bmatrix} = \begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{bmatrix} = \begin{bmatrix} 30 & 20 & 10 \\ 40 & 50 & 60 \\ 90 & 80 & 70 \end{bmatrix}$$



@tutorialgateway.org

C program to interchange Diagonals of a Matrix 1

Algorithm

Step 1: Declare two integer variables, 'row' and 'col'

Step 2: Read the size of the matrix in 'row' and 'col'

Step 3: Check if 'row' != 'col'. If this condition is true, display a warning message saying

“Please Enter a Square Matrix !!!” and move to step 2.

Step 4: Declare a two-dimensional matrix 'arr[row][col]' and a integer variable 'temp'

Step 5: Run a nested for loop from 'i' = '0' to 'row' and 'j' = '0' to 'col'

Step 6: Read the array elements from the user using the 'scanf' function.

Step 7: Run a for loop from 'i' = '0' to 'row'

Step 8: Swap the elements present on the diagonals using following code

- **temp = arr[i][i];**
- **arr[i][i] = arr[i][row - i - 1];**
- **arr[i][row - i - 1] = temp;**

Step 9: Run a nested for loop from 'i' = '0' to 'row' and 'j' = '0' to 'col'

Step 10: Display the matrix elements as 'arr [i] [j]'

Code

```
/* C Program to Interchange Diagonals of a Matrix */

#include<stdio.h>
int main()
{
    int row, col;

    label1:
    printf("\n Enter Number of rows and columns : ");
    scanf("%d %d", &row,&col);

    // Check if it is a square matrix or not
    if(row != col)
    {
        printf("\n Please Enter a Square Matrix !!!" );
        goto label1;
    }

    int arr[row][col], temp;
    printf("\n Enter the Matrix Elements \n");
    // Nested loop to input array elements from the user
    for(int i=0;i<row;i++)
    {
        for(int j=0;j<col;j++)
        {
            scanf("%d", &arr[i][j]);
        }
    }
}
```

```

    }

    // Code to swap the diagonals
    for(int i=0;i<row;i++)
    {
        temp = arr[i][i];
        arr[i][i] = arr[i][row - i - 1];
        arr[i][row - i - 1] = temp;
    }

    printf("\n Matrix after Interchanging Diagonals are: \n");
    // Nested loop to print the matrix after intercganing the diagonals
    for(int i=0;i<row;i++)
    {
        for(int j=0;j<col;j++)
        {
            printf(" %d  ", arr[i][j]);
        }
        printf("\n");
    }
    return 0;
}

```

Output

Example of 2x2 matrix:

```
Please Enter Number of rows and columns : 2 2
```

```
Please Enter the Matrix Elements
```

```
20  80
```

```
30  40
```

```
Matrix after Interchanging Diagonals are:
```

```
80  20
```

```
40  30
```

Example of 3x3 matrix:

```
Please Enter Number of rows and columns : 3 3
```

```
Please Enter the Matrix Elements
```

```
10  20  30
```

```
40  50  60
```

```
70  80  90
```

```
Matrix after Interchanging Diagonals are:
```

```
30  20  10
```

```
40  50  60
```

```
90  80  70
```

Time and Space C

5. Interchange any two Rows & Columns in the given Matrix.

Write a C Program to Swap rows and columns of matrix . Here's simple Program to Interchange any two Rows and Columns in Matrix in C Programming Language.

What is Matrix ?

Matrix representation is a method used by a computer language to store matrices of more than one dimension in memory. C uses “Row Major”, which stores all the elements for a given row contiguously in memory.

Original Matrix

1	2	3
4	5	6
7	8	9

The Matrix after swapping 2 Rows

1	2	3
4	5	6
7	8	9

Original Matrix

1	2	3
4	5	6
7	8	9

1	3	2
4	6	5
7	9	8

The Matrix after swapping 2 Columns

ALGORITHM

STEP 1: Include the header files to use the built-in functions in the C program.

STEP 2: Declare the matrix variables **m1[][]**, **m2[][]** as integer type.

STEP 3: Declare the integer variables **i,j,m,n,a,b,c,p,q,r**.

STEP 4: Read the Order of the Matrix into **m** and **n**.

STEP 5: Use a nested **for loop** with conditions **i and j** for reading elements of matrix **m1**, also assign the same value to matrix **m2**

STEP 6: Read the number of rows to be exchanged into the variables **a** and **b**.

STEP 7: By using **for loop** with the condition **i**, do step8

STEP 8: Swap the elements of rows as follows

1. $c = m1[a-1][i]$
2. $m1[a-1][i] = m1[b-1][i]$
3. $m1[b-1][i] = c.$

STEP 9: Read the number of columns to be exchanged into the variables **p** and **q**.

STEP 10: By using **for loop** display the given Matrix stored in **m2**, because **m1** has the row interchanged matrix.

STEP 11: Use a for loop with the condition **i**, do step12

STEP 12: Swap the elements of columns as follows

1. $r = m2[i][p-1]$
2. $m2[i][p-1] = m2[i][q-1]$
3. $m2[i][q-1] = r.$

STEP 13: Display the Matrix **m1** after interchanging the two Rows.

STEP 14: Display the Matrix **m2** after interchanging the two Columns.

C Source Code

```
#include <stdio.h>

void main() {

    static int m1[10][10], m2[10][10];

    int i, j, m, n, a, b, c, p, q, r;

    printf("Enter the order of the matrix\n");

    scanf("%d * %d", &m, &n);

    printf("Enter the coefficients of the matrix\n");

    for (i = 0; i < m; ++i) {

        for (j = 0; j < n; ++j) {

            scanf("%d", &m1[i][j]);

            m2[i][j] = m1[i][j];

        }

    }

    printf("Enter the numbers of two rows to be exchanged \n");
```

```

scanf("%d %d", &a, &b);
for (i = 0; i < m; ++i) {
    c = m1[a - 1][i]; /* first row has index is 0 */
    m1[a - 1][i] = m1[b - 1][i];
    m1[b - 1][i] = c;
}

printf("Enter the numbers of two columns to be exchanged\n");
scanf("%d %d", &p, &q);
printf("The given matrix is \n");
for (i = 0; i < m; ++i) {
    for (j = 0; j < n; ++j)
        printf(" %d", m2[i][j]);
    printf("\n");
}

for (i = 0; i < n; ++i) {
    r = m2[i][p - 1]; /* first column index is 0 */
    m2[i][p - 1] = m2[i][q - 1];
    m2[i][q - 1] = r;
}

printf("The matrix after interchanging the two rows(in the original matrix)\n");
for (i = 0; i < m; ++i) {
    for (j = 0; j < n; ++j) {
        printf(" %d", m1[i][j]);
    }
}

```

```
    printf("\n");
}
printf("The matrix after interchanging the two columns(in the original matrix)\n");
for (i = 0; i < m; ++i) {
    for (j = 0; j < n; ++j)
        printf(" %d", m2[i][j]);
    printf("\n");
}
}
```

Version 2

```
/* C Program to Swap rows and columns of matrix */
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    static int array1[10][10], array2[10][10];
```

```
    int i, j, m, n, a, b, c, p, q, r;
```

```
printf("Enter no. of rows :: ");

scanf("%d", &m);

printf("\nEnter no. of cols :: ");

scanf("%d",&n);

printf("\nEnter values to the matrix :: \n");

for (i = 0; i < m; i++)

{

    for (j = 0; j < n; j++)

    {

        printf("\nEnter a[%d][%d] value :: ",i,j);

        scanf("%d", &array1[i][j]);

        array2[i][j] = array1[i][j];

    }

}

printf("\nThe given matrix is \n\n");

for (i = 0; i < m; ++i)
```

```
{  
  
for (j = 0; j < n; ++j)  
  
{  
  
    printf("\t%d", array1[i][j]);  
  
}  
  
printf("\n\n");  
  
}
```

```
printf("\nEnter row number to be exchanged : ");  
  
scanf("%d", &a);  
  
printf("\nEnter other row number to be exchanged with : ");  
  
scanf("%d",&b);
```

```
for (i = 0; i < m; ++i)  
  
{  
  
    /* first row has index is 0 */
```

```
c = array1[a - 1][i];

array1[a - 1][i] = array1[b - 1][i];

array1[b - 1][i] = c;

}

printf("\nEnter col number to be exchanged : ");

scanf("%d", &p);

printf("\nEnter other col number to be exchanged with : ");

scanf("%d",&q);


for (i = 0; i < n; ++i)

{

    /* first column index is 0 */

    r = array2[i][p - 1];

    array2[i][p - 1] = array2[i][q - 1];

    array2[i][q - 1] = r;

}

printf("\nThe matrix after interchanging the two rows :: \n");
```

```
for (i = 0; i < m; ++i)

{

    for (j = 0; j < n; ++j)

    {

        printf("\t%d", array1[i][j]);

    }

    printf("\n\n");

}
```

```
printf("\nThe matix after interchanging the two columns :: \n");
```

```
for (i = 0; i < m; ++i)

{

    for (j = 0; j < n; ++j)

    {

        printf("\t%d", array2[i][j]);

    }

}
```

```
        printf("\n\n");

    }

    return 0;

}
```

Output ::

```
/* C Program to Swap rows and columns of matrix */
```

```
Enter no. of rows :: 3
```

```
Enter no. of cols :: 3
```

```
Enter values to the matrix ::
```

Enter a[0][0] value :: 1

Enter a[0][1] value :: 2

Enter a[0][2] value :: 3

Enter a[1][0] value :: 4

Enter a[1][1] value :: 5

Enter a[1][2] value :: 6

Enter a[2][0] value :: 7

Enter a[2][1] value :: 8

Enter a[2][2] value :: 9

The given matrix is

1	2	3
---	---	---

4	5	6
---	---	---

7	8	9
---	---	---

Enter row number to be exchanged : 2

Enter other row number to be exchanged with : 3

Enter col number to be exchanged : 1

Enter other col number to be exchanged with : 2

The matrix after interchanging the two rows ::

1	2	3
---	---	---

7	8	9
---	---	---

4	5	6
---	---	---

The matrix after interchanging the two columns ::

2	1	3
---	---	---

5	4	6
---	---	---

8	7	9
---	---	---

6. Search for an element in a given matrix and count the number of its occurrences.
C program to find the frequency of each element in the array

In this program, we have an array of elements to count the occurrence of its each element. One of the approaches to resolve this problem is to maintain one array to store the counts of each element of the array. Loop through the array and count the occurrence of each element as frequency and store it in another array fr.

1 2 8 3 2 2 2 5 1

In the given array, 1 has appeared two times so its frequency be 2 and 2 has appeared four times so have frequency 4 and so on.

ALGORITHM:

- **STEP 1:** START
- **STEP 2:** INITIALIZE arr[] = {1, 2, 8, 3, 2, 2, 2, 5, 1 }.
- **STEP 3:** length = sizeof(arr)/sizeof(arr[0])
- **STEP 4:** DEFINE fr[length].
- **STEP 5:** SET visited = -1.li>
- **STEP 6:** SET i= 0. REPEAT STEP 7 to STEP 12 UNTIL i<length.
- **STEP 7:** SET count = 1
- **STEP 8:** SET j =0. REPEAT STEP 9 and STEP 10 UNTIL j<length.
- **STEP**

9: if(arr[i]==arr[j])

then

count++
- fr[j] =visited
- **STEP 10:** j= j+1.
- **STEP**

11: if(fr[i]!=visited)

then
- fr[i]=count
- **STEP 12:** i=i+1
- **STEP 13:** PRINT "-----"

- **STEP 14:** PRINT "Element | Frequency"
- **STEP 15:** PRINT "-----"
- **STEP 16:** SET i=0. REPEAT STEP 17 to STEP 18 UNTIL i<length.
- **STEP17:** if(fr[i]!=visited)then
 PRINT arr[i]
 PRINT | PRINT fr[i]
- **STEP 18:** i=i+1.
- **STEP 19:** PRINT "-----"
- **STEP 20:** RETURN 0.
- **STEP 21:** END

C program to find occurrence of an element in one dimensional array

Example:

Input array elements:
 10, 10, 20, 30, 10
 Element to find occurrence: 10

Output:
 Occurrence of 10 is: 30

Program:

```
/* C program to find occurrence of an element
in one dimensional array.
*/

#include <stdio.h>
#define MAX 100

int main()
{
```

```

int arr[MAX], n, i;
int num, count;

printf("Enter total number of elements: ");
scanf("%d", &n);

//read array elements
printf("Enter array elements:\n");
for (i = 0; i < n; i++) {
    printf("Enter element %d: ", i + 1);
    scanf("%d", &arr[i]);
}

printf("Enter number to find Occurrence: ");
scanf("%d", &num);

//count occurrence of num
count = 0;
for (i = 0; i < n; i++) {
    if (arr[i] == num)
        count++;
}
printf("Occurrence of %d is: %d\n", num, count);
return 0;
}

```

Output:

```

Enter total number of elements: 5
Enter array elements:
Enter element 1: 10
Enter element 2: 10
Enter element 3: 20
Enter element 4: 30
Enter element 5: 10
Enter number to find Occurrence: 10
Occurrence of 10 is: 3

```

Write C program without using STRING-HANDLING functions to Count the number of words in a sentence.

Algorithm

1. Take a string as input and store it in the array of characters.
2. Create 3 counter variable for the count of words, lines and characters in the string.
3. Using for loop search for a space ' ' in the string and consecutively increment the variable count for words.
4. Using for loop search for a next line '\n' in the string and consecutively increment a variable count for next line.
5. Using for loop search for a characters except space and new line in the string and consecutively increment a variable count for characters.
6. Repeat step 3,4,5 until the loop reaches to the end of the string.
7. Check for the corner cases (discussed above) and do accordingly.
8. Print all the values of the counter.

Problem Solution

1. Take a string as input.
2. Using for loop search for a empty space in between the words in the string.
3. Consecutively increment a variable. This variable gives the count of number of words.

```
//C code
#include <stdio.h>

int main()
{
    char str[100]; //input string with size 100

    int words=0, newline=0, characters=0; // counter variables

    scanf("%[^\n]", &str); //scanf formatting

    for(int i=0; str[i]!='\0'; i++)
    {
```

```

    if(str[i] == ' ')
    {
        words++;
    }
    else if(str[i] == '\n')
    {
        newline++;
        words++;//since with every next line new words start. corner case 1
    }
    else if(str[i] != ' ' && str[i] != '\n'){
        characters++;
    }
}
if(characters > 0)//Corner case 2,3.
{
    words++;
    newline++;
}
printf("Total number of words : %d\n",words);
printf("Total number of lines : %d\n",newline);
printf("Total number of characters : %d\n",characters);
return 0;
}

```

Input : Deepak Kumar is the
brother of Aditya Kumar~

Output : Total number of words : 8

Total number of lines : 2

Total number of characters : 36

```

1. /** C Program to Count Number of Words in a given Text Or Sentence */
2. #include <stdio.h>
3. #include <string.h>
4. void main()
5. {
6.     char s[200];
7.     int count = 0, i;
8.     printf("Enter the string:\n");
9.     scanf("%[^\\n]s", s);
10.    for (i = 0; s[i] != '\\0'; i++)
11.    {
12.        if (s[i] == ' ' && s[i+1] != ' ')
13.            count++;
14.    }
15.    printf("Number of words in given string are: %d\\n", count + 1);
16. }

```

Write C programs without using STRING-HANDLING functions for the following:

1. Count the number of words in a sentence.

Write a program in C to count the total number of words in a string.

using Function

C

```

1 #include <stdio.h>
2 #include <string.h>
3
4 int stringwordcount(char *s)
5 {
6     int i, words=0;
7     for(i=0; s[i]; i++)
8     {
9         if(s[i]==32)
10            words++;
11     }
12     if(i>0)

```

```

13 words++;
14
15 return words;
16}
17int main()
18{
19
20 char s[1000];
21 int wordcount;
22
23 printf("Enter the string: ");
24 gets(s);
25
26
27 wordcount=stringwordcount(s);
28 printf("no of words in string = %d\n",wordcount);
29
30}
31
32
33

```

Version 2

```

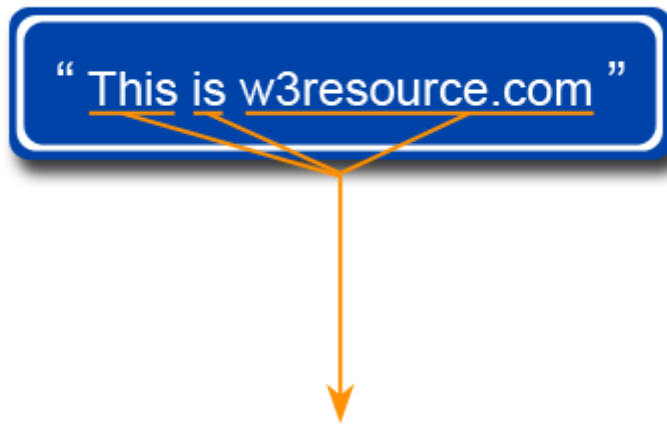
1. /*
2.  * C Program to Count Number of Words in a given Text Or Sentence
3.  */
4. #include <stdio.h>
5. #include <string.h>
6.
7. void main()
8. {
9.     char s[200];
10.    int count = 0, i;
11.
12.    printf("Enter the string:\n");
13.    scanf("%[^\\n]s", s);

```

```

14.  for (i = 0; s[i] != '\0'; i++)
15.  {
16.      if (s[i] == ' ' && s[i+1] != ' ')
17.          count++;
18.  }
19.  printf("Number of words in given string are: %d\n", count + 1);
20. }

```



Total number of words in the string is : 3

© w3resource.com

Sample Solution:

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define str_size 100 //Declare the maximum size of the string

void main()
{
    char str[str_size];

```

```

int i, wrd;

printf("\n\nCount the total number of words in a string :\n");

printf("-----\n");           printf("Input the string
: ");

fgets(str, sizeof str, stdin);

    i = 0;

wrd = 1;

/* loop till end of string */
while(str[i]!='\0')
{
    /* check whether the current character is white space or new line or tab character*/
    if(str[i]==' ' || str[i]=='\n' || str[i]=='\t')
    {
        wrd++;
    }

    i++;
}

printf("Total number of words in the string is : %d\n", wrd-1);
}

```

2. Input a string and toggle the case of every character in the input string.

Ex: INPUT: aBcDe

OUTPUT: AbCdE

```
/**
 * C program to toggle case of each character in a string
 */

#include <stdio.h>
#define MAX_SIZE 100 // Maximum string size

/* Toggle case function declaration */
void toggleCase(char * str);

int main()
{
    char str[MAX_SIZE];

    /* Input string from user */
    printf("Enter any string: ");
    gets(str);

    printf("String before toggling case: %s", str);

    toggleCase(str);

    printf("String after toggling case: %s", str);

    return 0;
}

/** * Toggle case of each character in given string */
void toggleCase(char * str)
{
    int i = 0;

    while(str[i] != '\0')
    {
        if(str[i]>='a' && str[i]<='z')
        {
            str[i] = str[i] - 32;
        }
    }
}
```

```

    }
    else if(str[i]>='A' && str[i]<='Z')
    {
        str[i] = str[i] + 32;
    }

```

```

    i++;
}

```

```

}

```

Using Recursion

C

```

1 #include <stdio.h>
2 #include <string.h>
3
4 int stringtogglecase(char *s)
5 {
6     static int i=0;
7     if(s[i])
8     {
9         if(s[i]>=65 && s[i]<=90)
10            s[i]+=32;
11        else if(s[i]>=97 && s[i]<=122)
12            s[i]-=32;
13
14        i++;
15        stringtogglecase(s);
16    }
17}
18int main()
19{
20    char s[1000];
21
22    printf("Enter the string: ");
23    gets(s);
24
25
26    stringtogglecase(s);
27
28    printf("string in togglecase = \"%s\\n\",s);

```

```
29  
30  
31 }
```

```
#include <stdio.h>  
#include <string.h>  
  
// Make sure that have knowledge about ASCII  
// if not please check - https://prepinsta.com/ascii-table/  
  
int main() {  
  
    char str[100];  
  
    printf("Please Enter any String: ");  
    gets(str);  
  
    for (int i = 0; str[i] != '\0'; i++) {  
  
        if (str[i] >= 'A' && str[i] <= 'Z')  
            str[i] = str[i] + 'a' - 'A';  
  
        else if (str[i] >= 'a' && str[i] <= 'z')  
            str[i] = str[i] + 'A' - 'a';  
    }  
  
    printf("Toggoled string: %s", str); // Print toggled string.  
    return 0;  
}
```

C Program To Toggle Case Of Each Character Of A String Using Standard Method
C

```
1 #include <stdio.h>  
2 #include <string.h>  
3  
4 int main()  
5 {  
6     char s[1000];
```

```

7  int i;
8
9  printf("Enter the string : ");
10 gets(s);
11
12
13
14  for(i=0;s[i];i++)
15  {
16      if(s[i]>=65 && s[i]<=90)
17          s[i]+=32;
18      else if(s[i]>=97 && s[i]<=122)
19          s[i]-=32;
20  }
21
22
23  printf("string in togglecase = '%s'\n",s);
24
25
26  return 0;
27}

```

Output:

```

1
2string in toggle case ='HELLO world'

```

```

C
1  #include <stdio.h>
2  #include <string.h>
3
4  void stringtogglecase(char *s)
5  {
6      int i;
7
8      for(i=0;s[i];i++)
9      {
10         if(s[i]>=65 && s[i]<=90)
11             s[i]+=32;
12         else if(s[i]>=97 && s[i]<=122)

```

```

13     s[i]-=32;
14     }
15
16
17 }
18 int main()
19 {
20
21     char s[1000];
22
23     printf("Enter the string: ");
24     gets(s);
25
26
27     stringtogglecase(s);
28
29     printf("string in togglecase ='%s'\n",s);
30
31 }

```

Check whether the given string is a palindrome or not.

```

/*
 * C program to find the length of a string without using the
 * built-in function also check whether it is a palindrome
 */
#include <stdio.h>
#include <string.h>

void main()
{
    char string[25], reverse_string[25] = {'\0'};

```

```

int i, length = 0, flag = 0;

printf("Enter a string \n");
gets(string);
/* keep going through each character of the string till its end */
for (i = 0; string[i] != '\0'; i++)
{
    length++;
}
printf("The length of the string '%s' = %d\n", string, length);
for (i = length - 1; i >= 0 ; i--)
{
    reverse_string[length - i - 1] = string[i];
}
/* Check if the string is a Palindrome */

for (flag = 1, i = 0; i < length ; i++)
{
    if (reverse_string[i] != string[i])
        flag = 0;
}
if (flag == 1)
    printf ("%s is a palindrome \n", string);
else

```

```
printf("%s is not a palindrome \n", string);
```

```
}
```

Example:

radar, madam, pop, lol, etc.

Palindrome String Check Program in C

Example:

```
#include <stdio.h>
#include <string.h>

int main(){
    char string1[20];
    int i, length;
    int flag = 0;

    printf("Enter a string:");
    scanf("%s", string1);

    length = strlen(string1);

    for(i=0;i < length ;i++){
        if(string1[i] != string1[length-i-1]){
            flag = 1;
            break;
        }
    }

    if (flag) {
        printf("%s is not a palindrome", string1);
    }
    else {
        printf("%s is a palindrome", string1);
    }
    return 0;
}
```

```
1 #include <stdio.h>
```

```
2 #include <string.h>
```

```

3
4 int main()
5 {
6     char s[1000];
7     int i,n,c=0;
8
9     printf("Enter the string : ");
10    gets(s);
11    n=strlen(s);
12
13    for(i=0;i<n/2;i++)
14    {
15        if(s[i]==s[n-i-1])
16            c++;
17
18    }
19    if(c==i)
20        printf("string is palindrome");
21    else
22        printf("string is not palindrome");
23
24
25
26    return 0;
27}

```

Arrange 'n' names in alphabetical order (hint: use string handling function-*strcpy*)

```

3. #include<stdio.h>
4. #include<string.h>
5. main(){
6.     int i,j,n;
7.     char str[100][100],s[100];
8.     printf("Enter number of names :
");

```

```

9.  scanf("%d",&n);
10. printf("Enter names in any order:
    ");
11. for(i=0;i<n;i++){
12.     scanf("%s",str[i]);
13. }
14. for(i=0;i<n;i++){
15.     for(j=i+1;j<n;j++){
16.         if(strcmp(str[i],str[j])>0){
17.             strcpy(s,str[i]);
18.             strcpy(str[i],str[j]);
19.             strcpy(str[j],s);
20.         }
21.     }
22. }
23. printf("
    The sorted order of names are:
    ");
24. for(i=0;i<n;i++){
25.     printf("%s
        ",str[i]);
26. }
27. }

```

28. Output

29. When the above program is executed, it produces the following result –

30. Enter number of names:

31. 5

32. Enter names in any order:

33. Pinky

34. Lucky
35. Ram
36. Appu
37. Bob
38. The sorted order of names is:
39. Appu
40. Bob
41. Lucky
42. Pinky
43. Ram

Version 2

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
int main()
{
    char str[5][20], t[20];
    int i, j;
    printf("Enter any string (5 names):\n");
    for(i=0; i<5; i++)
        scanf("%s", str[i]);
    for(i=1; i<5; i++)
    {
        for(j=1; j<5; j++)
        {
            if(strcmp(str[j-1], str[j])>0)
            {
                strcpy(t, str[j-1]);
                strcpy(str[j-1], str[j]);
                strcpy(str[j], t);
            }
        }
    }
    printf("\nStrings (Names) in alphabetical order:\n");
    for(i=0; i<5; i++)
```

```

    printf("%s\n", str[i]);
    getch();
    return 0;
}

```

rite a program in C to read a string through keyboard and sort it using bubble sort.

Sort it using bubble sort



© w3resource.com

Sample Solution:

C Code:

```

#include <stdio.h>

#include <string.h>

void main()
{
    char name[25][50],temp[25];

```

```

int n,i,j;

printf("\n\nSorts the strings of an array using bubble sort :\n");
printf("-----\n");

printf("Input number of strings :");
scanf("%d",&n);

printf("Input string %d :\n",n);
for(i=0;i<=n;i++)
{

    fgets(name[i], sizeof name, stdin);
}

/*Logic Bubble Sort*/

for(i=1;i<=n;i++)
    for(j=0;j<=n-i;j++)
        if(strcmp(name[j],name[j+1])>0)
        {

```

```

        strcpy(temp,name[j]);

        strcpy(name[j],name[j+1]);

        strcpy(name[j+1],temp);

    }

    printf("The strings appears after sorting :\n");

    for(i=0;i<=n;i++)

        printf("%s\n",name[i]);

}

```

Copy

Sample Output:

Sorts the strings of an array using bubble sort :

 Input number of strings :3

Input string 3 :

zero

one

two

The strings appears after sorting :

one

two

zero

Flowchart :

5. Delete a word from the given sentence.

Ex: INPUT: I AM STUDYING IN MIT

TO BE DELETED: STUDYING

```
#include <stdio.h>
int search(char[], char[]);
int delete_word(char[], char[],
int);
int main()
{
    char str[80], word[50];
    int index;
    printf("Enter string:\n");
    gets(str);
    printf("Enter word to
delete:\n");
    gets(word);
    index = search(str, word);
    if (index != - 1)
    {
        delete_word(str, word,
index);
        printf("String without
word:\n%s", str);
    }
    else
    {
        printf("The word not
present in the string.");
    }

    return 0;
}
```

```

    /* Function returns the index of
str where word is found */
    int search(char str[], char
word[])
    {
        int l, i, j;

        /* finding length of word */
        for (l = 0; word[l] != '\0'; l++);
        for (i = 0, j = 0; str[i] !=
'\0' && word[j] != '\0'; i++)
        {
            if (str[i] == word[j])
            {
                j++;
            }
            else
            {
                j = 0;
            }
        }
        if (j == l)
        {
            /* substring found */
            return (i - j);
        }
        else
        {
            return - 1;
        }
    }

    int delete_word(char str[], char

```

```

word[], int index)
{
    int i, l;
    /* finding length of word */
    for (l = 0; word[l] != '\0'; l++);

    for (i = index; str[i] != '\0';
i++)
    {
        str[i] = str[i + l + 1];
    }
}

```

```

#include<stdio.h>
#include<conio.h>
#include<string.h>
int main()
{
    char str[100], word[20];
    int i, j, ls, lw, temp, chk=0;
    printf("Enter the String: ");
    gets(str);
    printf("Enter a Word: ");
    gets(word);
    ls = strlen(str);
    lw = strlen(word);
    for(i=0; i<ls; i++)
    {
        temp = i;
        for(j=0; j<lw; j++)
        {
            if(str[i]==word[j])
                i++;
        }
        chk = i-temp;
        if(chk==lw)
        {

```

```

    i = temp;
    for(j=i; j<(ls-lw); j++)
        str[j] = str[j+lw];
    ls = ls-lw;
    str[j]='\0';
}
}
printf("\nNew String = %s", str);
getch();
return 0;
}

```

Module5:MODULAR PROGRAMMING-Functions & RECURSIVE Functions

Write a C program using function “Largest” to find the maximum of a given list of numbers. Also write a main program to read N numbers and find the largest among them using this function.

```

// C program to find maximum in arr[] of size n
#include <stdio.h>

```

```

// C function to find maximum in arr[] of size n

```

```

int largest(int arr[], int n)

```

```

{

```

```

    int i;

```

```

    // Initialize maximum element

```

```

    int max = arr[0];

```

```

    // Traverse array elements from second and

```

```

    // compare every element with current max

```

```

    for (i = 1; i < n; i++)

```

```

        if (arr[i] > max)

```

```

            max = arr[i];

```

```

    return max;

```

```

}

```

```

int main()
{
    int arr[] = {10, 324, 45, 90, 9808};
    int n = sizeof(arr)/sizeof(arr[0]);
    printf("Largest in given array is %d", largest(arr, n));
    return 0;
}

```

Output

Largest in given array is 9808

/ C program to find maximum in arr[] of size n

#include <stdio.h>

// C function to find maximum in arr[] of size n

```

int largest(int arr[], int n)

```

```

{
    int i;

    // Initialize maximum element
    int max = arr[0];

```

// Traverse array elements from second and

// compare every element with current max

```

for (i = 1; i < n; i++)

```

```

    if (arr[i] > max)
        max = arr[i];

```

```

    return max;

```

```

}

```

```

int main()

```

```

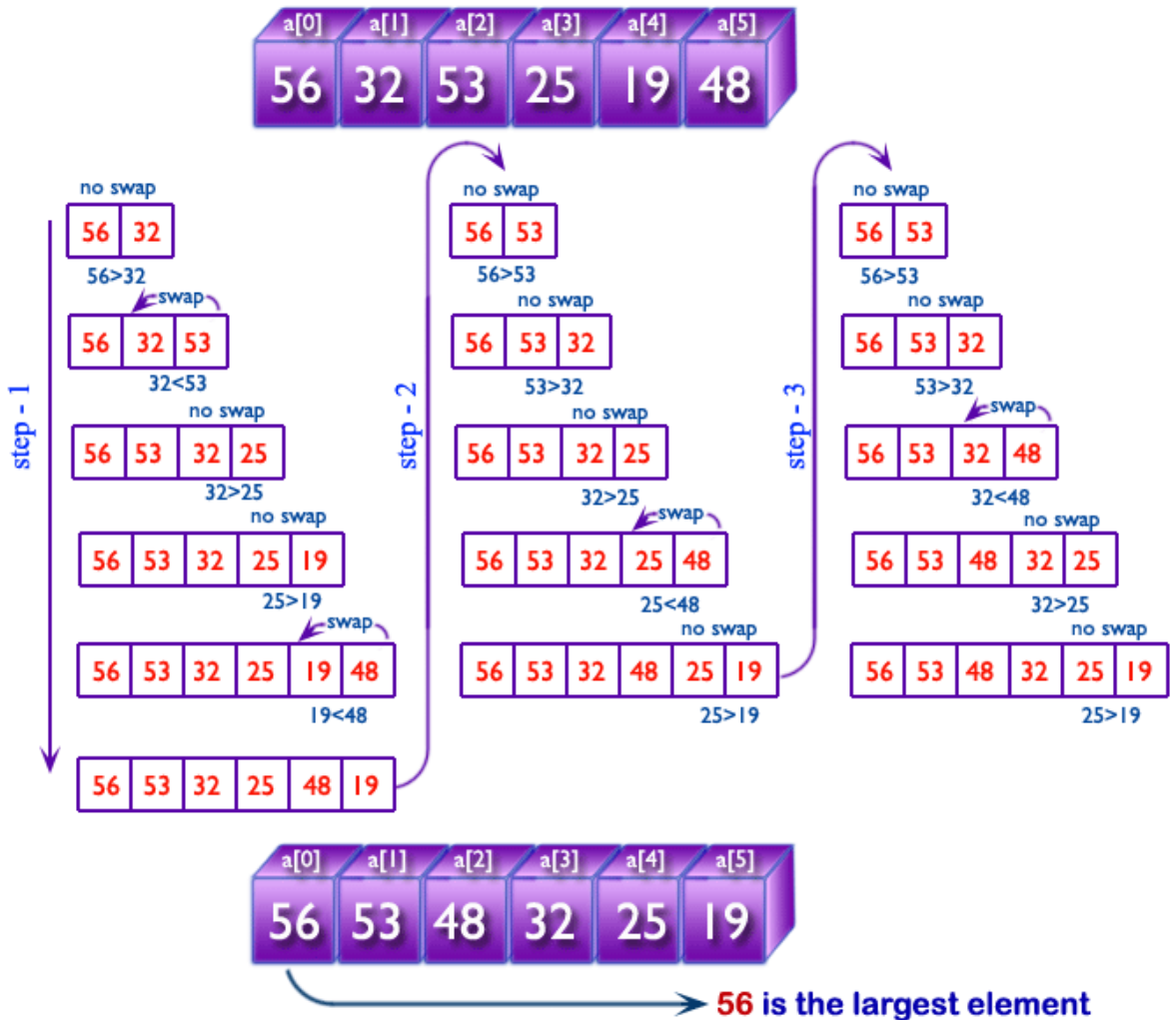
{
    int arr[] = {10, 324, 45, 90, 9808};
    int n = sizeof(arr)/sizeof(arr[0]);
    printf("Largest in given array is %d", largest(arr, n));
    return 0;
}

```

Write a program in C to get the largest element of an array using recursion.

Pictorial Presentation:

Largest element in an array



© w3resource.com

Sample Solution:

C Code:

```

#include<stdio.h>

#define MAX 100

int MaxElem(int []);

int n;

int main()
{
    int arr1[MAX],hstno,i;

    printf("\n\n Recursion : Get the largest element of an array :\n");
    printf("-----\n");

    printf(" Input the number of elements to be stored in the array :");
    scanf("%d",&n);

    printf(" Input %d elements in the array :\n",n);
    for(i=0;i<n;i++)
    {
        printf(" element - %d : ",i);
        scanf("%d",&arr1[i]);
    }
}

```

```

    hstno=MaxElem(arr1);//call the function MaxElem to return the largest element

    printf(" Largest element of the array is: %d\n\n",hstno);

    return 0;

}

int MaxElem(int arr1[])
{
    static int i=0,hstno =-9999;

    if(i < n)
    {
        if(hstno<arr1[i])

            hstno=arr1[i];

        i++;

        MaxElem(arr1);//calling the function MaxElem itself to compare with further element
    }

    return hstno;
}

```

Copy

Sample Output:

Recursion : Get the largest element of an array :

Input the number of elements to be stored in the array :5

Input 5 elements in the array :

element - 0 : 5

```
element - 1 : 10
element - 2 : 15
element - 3 : 20
element - 4 : 25
Largest element of the array is: 25
```

Write a C recursive function to find the factorial .

```
1. #include<stdio.h>
2.
3. long factorial(int n)
4. {
5.     if (n == 0)
6.         return 1;
7.     else
8.         return(n * factorial(n-1));
9. }
10.
11. void main()
12. {
13.     int number;
14.     long fact;
15.     printf("Enter a number: ");
16.     scanf("%d", &number);
17.
18.     fact = factorial(number);
19.     printf("Factorial of %d is %ld\n", number, fact);
20.     return 0;
21. }
```

Output:

```
Enter a number: 6
```

Factorial of 5 is: 720

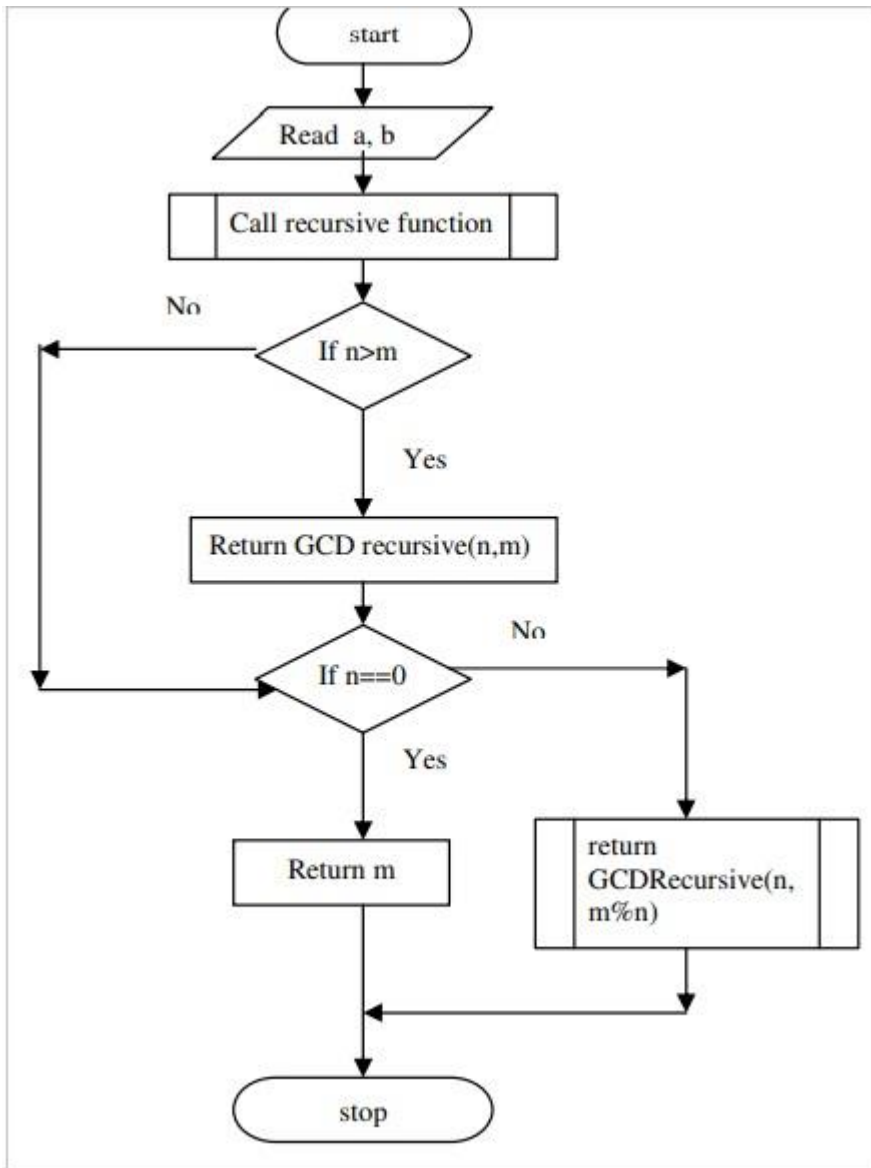
```
/* Program Name: Find Factorial
   Written by: Chaitanya Singh
   Published on: beginnersbook.com
*/
#include<stdio.h>
int find_factorial(int);
int main()
{
    int num, fact;
    //Ask user for the input and store it in num
    printf("\nEnter any integer number:");
    scanf("%d",&num);

    //Calling our user defined function
    fact =find_factorial(num);

    //Displaying factorial of input number
    printf("\nfactorial of %d is: %d",num, fact);
    return 0;
}
int find_factorial(int n)
{
    //Factorial of 0 is 1
    if(n==0)
        return(1);

    //Function calling itself: recursion
    return(n*find_factorial(n-1));
}
```

Write a recursive function, GCD to find the GCD of two numbers. Write a c recursive main program which reads 2 numbers and finds the GCD of the numbers using the specified function. Ex: GCD of 9, 24 is 3.



```

include<stdio.h>
#include<math.h>
unsigned int GCD(unsigned i, unsigned j);
int main(){

```

```

int a,b;
printf("Enter the two integers: ");
scanf("%d%d",&a,&b);
printf("GCD of %d and %d is %d",a,b,GCD(a,b));
return 0;
}
/* Recursive Function*/
unsigned int GCD(unsigned i, unsigned j){
    if(j>i)
        return GCD(j,i);
    if(j==0)
        return i;
    else
        return GCD(j,i%j);
}

```

Output

When the above program is executed, it produces the following result –

```

Enter the two integers: 4 8
GCD of 4 and 8 is 4

```

```

#include <stdio.h>
int hcf(int n1, int n2);
int main() {
    int n1, n2;
    printf("Enter two positive integers: ");
    scanf("%d %d", &n1, &n2);
    printf("G.C.D of %d and %d is %d.", n1, n2, hcf(n1, n2));
    return 0;
}

```

```
int hcf(int n1, int n2) {
    if (n2 != 0)
        return hcf(n2, n1 % n2);
    else
        return n1;
}
```

Enter two positive integers: 366
60
G.C.D of 366 and 60 is 6.

2. Write a c program Using a recursive function FIB to generate nth Fibonacci term. Write a main program to print first N Fibonacci terms using function FIB. [Hint: Fibonacci series is 0, 1, 1, 2, 3, 5, 8 ...]

Logic to find nth Fibonacci term using recursion

$$f(n) = \begin{cases} 0, & n = 0 \\ 1, & n = 1 \\ f(n-1) + f(n-2), & n > 1 \end{cases}$$

The recursive function to find n^{th} Fibonacci term is based on below three conditions.

1. If `num == 0` then `return 0`. Since Fibonacci of 0^{th} term is 0.
2. If `num == 1` then `return 1`. Since Fibonacci of 1^{st} term is 1.
3. If `num > 1` then `return fibo(num - 1) + fibo(n-2)`. Since Fibonacci of a term is sum of previous two terms.

Program to find nth Fibonacci term using recursion

```
/**
 * C program to find nth Fibonacci term using recursion
 */

#include <stdio.h>

/* Function declaration */
unsigned long long fibo(int num);

int main()
{
    int num;
    unsigned long long fibonacci;

    /* Input a number from user */
    printf("Enter any number to find nth fiboacci term: ");
    scanf("%d", &num);

    fibonacci = fibo(num);

    printf("%d fibonacci term is %llu", num, fibonacci);

    return 0;
}

/**
 * Recursive function to find nth Fibonacci term
 */
unsigned long long fibo(int num)
{
    if(num == 0)    //Base condition
        return 0;
    else if(num == 1) //Base condition
        return 1;
    else
```

```
return fibo(num-1) + fibo(num-2);
```

```
}
```

Output

Enter any number to find nth fiboacci term: 10

10 fibonacci term is 55

Version 2

```
1. /*
2.  * C Program to find the nth number in Fibonacci series using recursion
3.  */
4. #include <stdio.h>
5. int fibo(int);
6.
7. int main()
8. {
9.     int num;
10.    int result;
11.
12.    printf("Enter the nth number in fibonacci series: ");
13.    scanf("%d", &num);
14.    if (num < 0)
15.    {
16.        printf("Fibonacci of negative number is not possible.\n");
17.    }
18.    else
19.    {
20.        result = fibo(num);
21.        printf("The %d number in fibonacci series is %d\n", num, result);
22.    }
23.    return 0;
24. }
25. int fibo(int num)
26. {
27.     if (num == 0)
28.     {
29.         return 0;
30.     }
31.     else if (num == 1)
```

```

32.  {
33.      return 1;
34.  }
35.  else
36.  {
37.      return(fibo(num - 1) + fibo(num - 2));
38.  }
39. }

```

\$ **cc** pgm9.c

\$ a.out

Enter the nth number **in** fibonacci series: 8

The 8 number **in** fibonacci series is 21

\$ a.out

Enter the nth number **in** fibonacci series: 12

The 12 number **in** fibonacci series is 144

//Fibonacci Series using Dynamic Programming

```
#include<stdio.h>
```

```
int fib(int n)
```

```
{
```

```
    /* Declare an array to store Fibonacci numbers. */
```

```
    int f[n+2]; // 1 extra to handle case, n = 0
```

```
    int i;
```

```
    /* 0th and 1st number of the series are 0 and 1*/
```

```
    f[0] = 0;
```

```
    f[1] = 1;
```

```
    for (i = 2; i <= n; i++)
```

```
    {
```

```
        /* Add the previous 2 numbers in the series
        and store it */
```

```
        f[i] = f[i-1] + f[i-2];
```

```

    }

    return f[n];
}

int main ()
{
    int n = 9;
    printf("%d", fib(n));
    getchar();
    return 0;
}

```

3. Find out the length of string using pointer.

Program : Length of the String using Pointer

```

1  #include<stdio.h>
2  #include<conio.h>
3
4  int string_ln(char*);
5
6  void main() {
7      char str[20];
8      int length;
9      clrscr();
10
11     printf("\nEnter any string : ");
12     gets(str);
13
14     length = string_ln(str);
15     printf("The length of the given string %s is : %d", str, length);
16     getch();
17 }
18
19 int string_ln(char*p) /* p=&str[0] */
20 {

```

```
21 int count = 0;
22 while (*p != '\0') {
23     count++;
24     p++;
25 }
26 return count;
27 }
```

Output :

```
1 Enter the String : pritesh
2 Length of the given string pritesh is : 7
```

```
#include<stdio.h>

int main() {
    char str[20], *pt;
    int i = 0;
    printf("Pointer Example Program : Find or Calculate Length of String \n");
    printf("Enter Any string [below 20 chars] : ");
    gets(str);
    pt = str;
    while (*pt != '\0') {
        i++;
        pt++;
    }
    printf("Length of String : %d", i);

    return 0;
}
```

```
}
```

WAP to create the structure containing student name & marks of 3 subjects and find out & print total & average scored by the no. of students

Problem

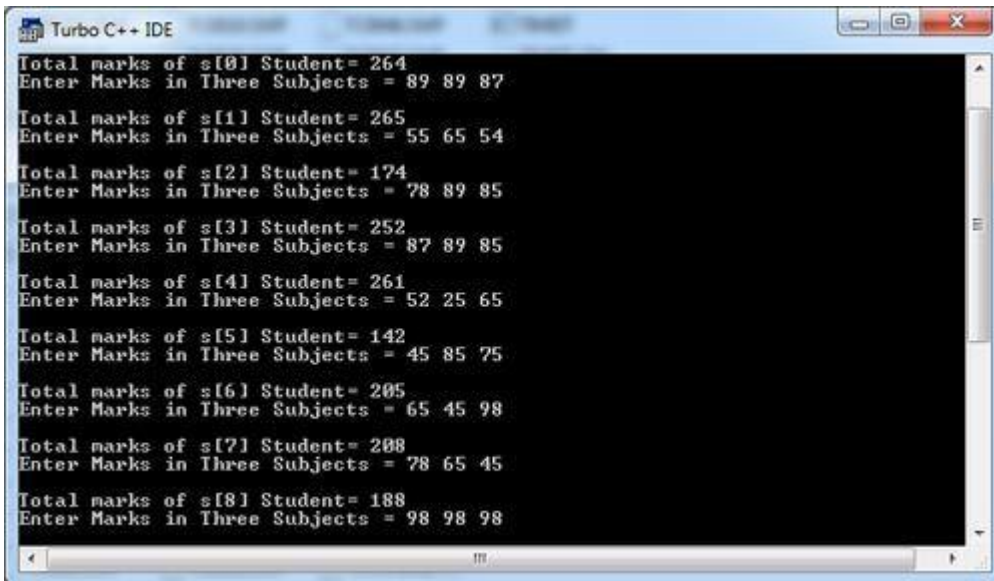
Statement:

The annual examination is conducted for 10 students for three subjects. Write a program to read the data and determine the following:

- (a) Total marks obtained by each student.
- (b) The highest marks in each subject and the marks of the student who secured it.
- (c) The student who obtained the highest total marks.

Here is source code of the C program to calculate The Marks and the grades of Students . The C program is successfully compiled. The program output is also shown below.

```
#include<stdio.h>
struct student
{
    int sub1;
    int sub2;
    int sub3;
};
void main()
{
    struct student s[10];
    int i,total=0;
    clrscr();
    for(i=0;i<=2;i++)
    {
        printf("\nEnter Marks in Three Subjects = ");
        scanf("%d%d%d",& s[i].sub1,&s[i].sub2,&s[i].sub3);
        total=s[i].sub1+s[i].sub2+s[i].sub3;
        printf("\nTotal marks of s[%d] Student= %d",i,total);
    }
    getch();
}
```



```
Turbo C++ IDE
Total marks of s[0] Student= 264
Enter Marks in Three Subjects = 89 89 87
Total marks of s[1] Student= 265
Enter Marks in Three Subjects = 55 65 54
Total marks of s[2] Student= 174
Enter Marks in Three Subjects = 78 89 85
Total marks of s[3] Student= 252
Enter Marks in Three Subjects = 87 89 85
Total marks of s[4] Student= 261
Enter Marks in Three Subjects = 52 25 65
Total marks of s[5] Student= 142
Enter Marks in Three Subjects = 45 85 75
Total marks of s[6] Student= 205
Enter Marks in Three Subjects = 65 45 98
Total marks of s[7] Student= 208
Enter Marks in Three Subjects = 78 65 45
Total marks of s[8] Student= 188
Enter Marks in Three Subjects = 98 98 98
```

Store Information and Display it Using Structure

```
#include <stdio.h>
struct student {
    char name[50];
    int roll;
    float marks;
} s;

int main() {
    printf("Enter information:\n");
    printf("Enter name: ");
    fgets(s.name, sizeof(s.name), stdin);

    printf("Enter roll number: ");
    scanf("%d", &s.roll);
    printf("Enter marks: ");
    scanf("%f", &s.marks);

    printf("Displaying Information:\n");
    printf("Name: ");
```

```
printf("%s", s.name);  
printf("Roll number: %d\n", s.roll);  
printf("Marks: %.1f\n", s.marks);  
  
return 0;  
}  
Run Code
```

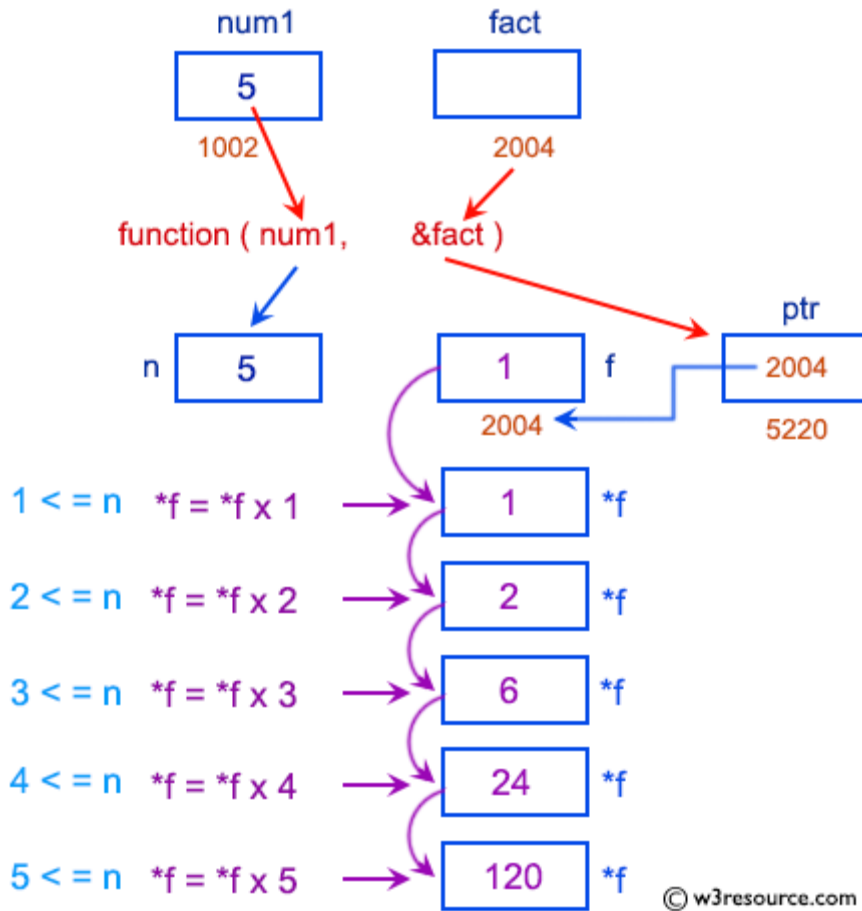
Output

```
Enter information:  
Enter name: Jack  
Enter roll number: 23  
Enter marks: 34.5  
Displaying Information:  
Name: Jack  
Roll number: 23  
Marks: 34.5
```

Write a Cprogram using function “Fact” to find the factorial of a given number, using pointer. Using this function, and compute NCR in the main function.

Write a program in C to find the factorial of a given number using pointers.

Pictorial Presentation:



Sample Solution:

C Code:

```
#include <stdio.h>

void findFact(int,int*);

int main()
{
```

```

int fact;

int num1;

printf("\n\n Pointer : Find the factorial of a given number :\n");
printf("-----\n");
printf(" Input a number : ");
scanf("%d",&num1);

findFact(num1,&fact);

printf(" The Factorial of %d is : %d \n\n",num1,fact);

return 0;
}

```

```

void findFact(int n,int *f)
{

int i;

*f=1;

for(i=1;i<=n;i++)

*f=*f*i;

}

```

Copy

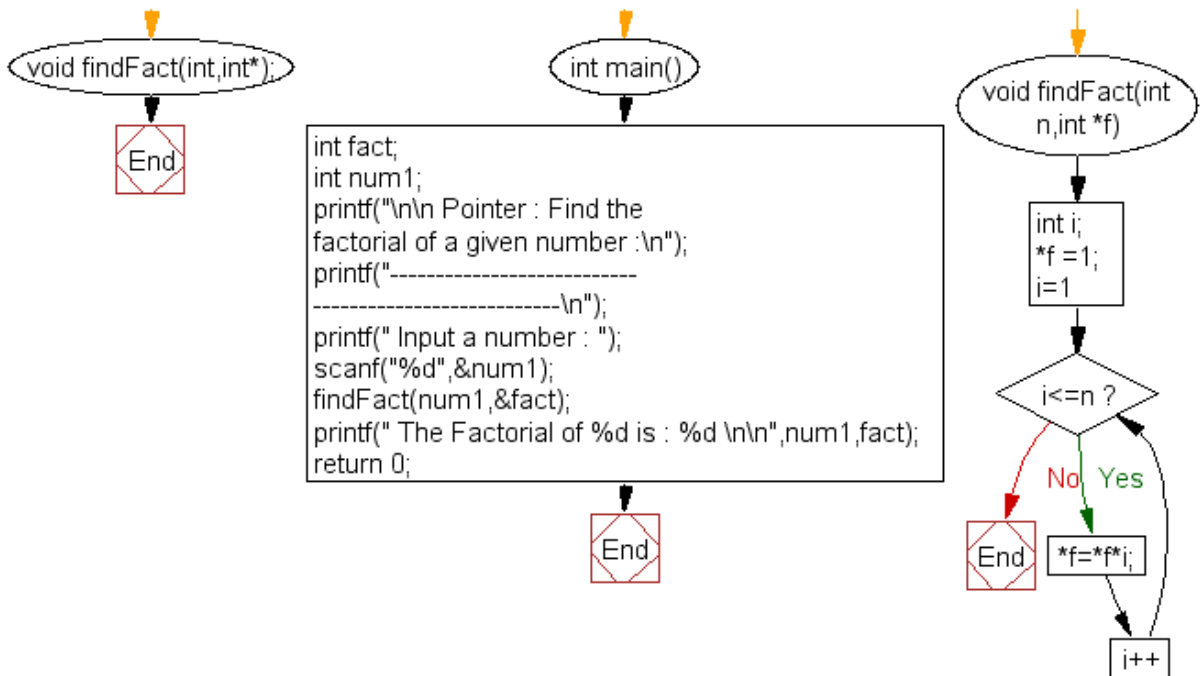
Sample Output:

Pointer : Find the factorial of a given number :

Input a number : 5

The Factorial of 5 is : 120

Flowchart:



```
int fact(int *a)
{
    int factorial=1,b;
    for(b=*a;b>0;b--){
```

```

        factorial*=b;
    }
    return factorial;
}

int main()
{
    int num, factorial;
    printf("please enter a number:");
    scanf("%d",&num);
    factorial = fact(&num);
    printf("%d",factorial);

    return 0;
}

```

Write a C program using a pointer to function “Largest” to find the maximum of a given list of numbers. Also write a main program to read N numbers and find the largest among them using this function.

```

#include <stdio.h>

int main(){
    int num1, num2, num3;
    int *p1, *p2, *p3;
    printf("enter 1st no: ");
    scanf("%d",&num1);
    printf("enter 2nd no: ");
    scanf("%d",&num2);
    printf("enter 3rd no: ");

```

```

scanf("%d",&num3);
p1 = &num1;
p2 = &num2;
p3 = &num3;
if(*p1 > *p2){
    if(*p1 > *p3){
        printf("%d is largest ", *p1);
    }else{
        printf("%d is largest ", *p3);
    }
}else{
    if(*p2 > *p3){
        printf("%d is largest ", *p2);
    }else{
        printf("%d is largest ", *p3);
    }
}
return 0;
}

```

Output

When the above program is executed, it produces the following result –

Run 1:

enter 1st no: 35

enter 2nd no: 75

enter 3rd no: 12

75 is largest

Run 2:

enter 1st no: 53

enter 2nd no: 69

enter 3rd no: 11

69 is largest

Example: Find the Largest Element

```
#include <stdio.h>
#include <stdlib.h>

int main() {

    int n;
    double *data;
    printf("Enter the total number of elements: ");
    scanf("%d", &n);

    // Allocating memory for n elements
    data = (double *)calloc(n, sizeof(double));
    if (data == NULL) {
        printf("Error!!! memory not allocated.");
        exit(0);
    }

    // Storing numbers entered by the user.
    for (int i = 0; i < n; ++i) {
        printf("Enter number%d: ", i + 1);
        scanf("%lf", data + i);
    }

    // Finding the largest number
    for (int i = 1; i < n; ++i) {
        if (*data < *(data + i)) {
            *data = *(data + i);
        }
    }
}
```

```
printf("Largest number = %.2lf", *data);

free(data);

return 0;
}
```

[Run Code](#)

Output

```
Enter the total number of elements: 5
Enter number1: 3.4
Enter number2: 2.4
Enter number3: -5
Enter number4: 24.2
Enter number5: 6.7
Largest number = 24.20
```

Write a C program using the pointers to function “IsPalin” to check whether the given string is a palindrome or not. Write a main function to test this function.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int checkpalindrome(char *s)
```

```
{
```

```
    int i,c=0,n;
```

```
    n=strlen(s);
```

```
    for(i=0;i<n/2;i++)
```

```
    {
```

```
        if(s[i]==s[n-i-1])
```

```
        c++;
```

```
    }
```

```
    if(c==i)
```

```
        return 1;
```

```
    else
```

```
        return 0;
```

```

}
int main()
{
    char s[1000];

    printf("Enter the string: ");
    gets(s);

    if(checkpalindrome(s))
        printf("string is palindrome");
    else
        printf("string is not palindrome");

}

```

Output:

```

1
2string is palindrome

```

Using Recursion

1. The main() calls the function checkpalindrome(char *s).
- 2) The function checkpalindrome(char * s)i=0,c=0.Calculate the string length n using strlen(s).
 - a) i=0,c=0.Calculate the string length n using strlen(s).
 - b) if i<length of the string/2
If the element at s[i] is equal to the element at s[n-i-1] then increase the c value and i value.
The function calls itself.
The function calls itself recursively until i<n/2.
 - c) If i!=length of the string/2,
If i=c then this function returns 1 otherwise it returns 0.
- 3) If the returned value is 1 then print the given string is a palindrome. If the returned value is 0 then print the string is not a palindrome.

Using Recursion

C

```

1 #include <stdio.h>
2 #include <string.h>
3
4
5 int checkpalindrome(char *s)
6 {
7     static int i,c=0,n=strlen(s);
8     if(i<n/2)
9     {
10         if(s[i]==s[n-i-1])
11             c++;
12             i++;
13         checkpalindrome(s);
14     }
15     else
16     {
17         if(c==i)
18             return 1;
19         else
20             return 0;
21     }
22
23
24 }
25 int main()
26 {
27     char s[1000];
28
29     printf("Enter the string: ");
30     gets(s);
31
32
33     if(checkpalindrome(s))
34         printf("string is palindrome");
35     else
36         printf("string is not palindrome");
37
38 }

```

Given an array of size 2X2 and the challenge is to print the sum of all the corner elements stored in an array.

Assume a matrix $\text{mat}[r][c]$, with some row "r" and column "c" starting row and column from 0, then its corner elements will be; $\text{mat}[0][0]$, $\text{mat}[0][c-1]$, $\text{mat}[r-1][0]$, $\text{mat}[r-1][c-1]$. Now the task is to get these corner elements and sum those corner elements i.e., $\text{mat}[0][0] + \text{mat}[0][c-1] + \text{mat}[r-1][0] + \text{mat}[r-1][c-1]$, and print the result on the screen.

Example

Input: Enter the matrix elements :

10 2 10

2 3 4

10 4 10

Output: sum of matrix is : 40

10	2	10
2	3	4
10	4	10

$$\begin{aligned}\text{sum} &= 10+10+10+10 \\ &= 40\end{aligned}$$

Algorithm

START

Step 1-> create macro for rows and column as `#define row 3` and `#define col 3`

Step 2 -> `main()`

Declare `int sum=0` and array as `a[row][col]` and variables `int i,j,n`

Loop For `i=0` and `i<3` and `i++`

```
Loop For j=0 and j<3 and j++
    Input a[i][j]
End
End
Print [0][0] + a[0][row-1] +a[col-1][0] + a[col-1][row-1]
STOP
```

Example

```
#include<stdio.h>
#define row 3
#define col 3
int main(){
    int sum=0,a[row][col],i,j,n;
    printf("Enter the matrix elements : ");
    for(i=0;i<3;i++){
        for(j=0;j<3;j++){
            scanf("%d",&a[i][j]);
        }
    }
    printf("sum of matrix is : %d",a[0][0] + a[0][row-1] +a[col-1][0] + a[col-1][row-1] );
    return 0;
}
```

Output

if we run above program then it will generate following output

Enter the matrix elements :

10 2 10

2 3 4

10 4 10

sum of matrix is : 40

1. To Learn the concept of recursion and to write recursive functions
2. Declare and initialize pointer variable
3. Write basic operations and programs using structures
 - 4. Access a variable through its pointer • A recursive function is a function that invokes/calls itself directly or indirectly.
 - • A Pointer is a memory location or a variable which stores the address of another variable in memory
 - • A structure in the C programming language (and many derivatives) is a composite data type (or record) declaration that defines a physically grouped list of variables to be placed under one name in a block of memory.
 - • A file is a place on disc where group of related data is stored.
 -
 -

MODULAR PROGRAMMING-RECURSIVE FUNCTIONS, STRUCTURES, POINTERS

Objectives:

In this lab, student will be able to:

Introduction:

Steps to Design a Recursive Algorithm

- **Base case:** ▪ for a small value of n , it can be solved directly
- Smaller versions of the same problem
- Identify the base case and provide a solution to it
- Reduce the problem to smaller versions of itself
- Move towards the base case using smaller versions
- **Recursive case(s)**
- The asterisk (*) tells the variable `pt_name` is a pointer variable.

- pt_name needs a memory location.
- pt_name points to a variable of type data_type

Solved Exercise

Program to explain the concept of recursive functions

```
#include<stdio.h>
long factorial (long a) { if (a ==0) //base case
return (1);
return (a * factorial (a-1));
}
int main () { long number;
printf("Please type a number: "); scanf("%d",&number);
printf("%d factorial is %ld",number, factorial (number)); return 0;
}
```

Declaring and initializing pointers: Syntax:

```
data_type * pt_name;
```

This tells the compiler 3 things about the pt_name:

Solved Exercise: #include<stdio.h> int main()

```
{
int var1 = 11; //two integer variables int var2 = 22;
int *ptr; //pointer to integer
ptr = &var1; //pointer points to var1 printf("%d",*ptr); //print contents of pointer (11) ptr =
&var2; //pointer points to var2 printf("%d",*ptr); //print contents of pointer (22)
return 0;
}
```

Declaration and initialization of structures: Déclaration :

```
struct student
{ int rollno, age; char name[20];
}s1, s2, s3;
```

Initialisation :

```
int main( ){ struct
{ int rollno; int age;
}stud={20, 21};
...
... return 0;
}
```

Solved Exercise: #include<stdio.h>
struct Book{ //Structure Definition
char title[20]; char author[15]; int pages;
float price; 30

```

};
int main( ){
struct Book b[10]; int i,j;
printf("Input values"); for (i=0;i<10;i++){
scanf("%s %s %d %f",b[i].title,b[i].author,&b[i].pages,&b[i].price);
}
for (j=0;j<10;j++){
printf("title %s\n author %s\n pages %d\n
price%f\n",b[j].title,b[j].author,b[j].pages,b[j].price);
}
return 0;
}

```

Find the maximum number in the input integer array using pointers.

```

1. #include <stdio.h>
2.
3. int main()
4. {
5.     long array[100], *maximum, size, c, location = 1;
6.
7.     printf("Enter the number of elements in array\n");
8.     scanf("%ld", &size);
9.
10.    printf("Enter %ld integers\n", size);
11.
12.    for ( c = 0 ; c < size ; c++ )
13.        scanf("%ld", &array[c]);
14.
15.    maximum = array;
16.    *maximum = *array;
17.
18.    for (c = 1; c < size; c++)
19.    {
20.        if (*(array+c) > *maximum)
21.        {
22.            *maximum = *(array+c);
23.            location = c+1;

```

```
24. }
25. }
26.
27. printf("Maximum element is present at location number %ld and it's value is
    %ld.\n", location, *maximum);
28. return 0;
29. }
```

**Create a student record with name, rollno, marks of 3 subjects (m1, m2, m3).
Compute the average of marks for 3 students and display the names of the students
in ascending order of their average marks.**

```
#include <stdio.h>
struct student {
    char name[50];
    int roll;
    float marks;
} s;

int main() {
    printf("Enter information:\n");
    printf("Enter name: ");
    fgets(s.name, sizeof(s.name), stdin);

    printf("Enter roll number: ");
    scanf("%d", &s.roll);
    printf("Enter marks: ");
    scanf("%f", &s.marks);

    printf("Displaying Information:\n");
```

```
printf("Name: ");
printf("%s", s.name);
printf("Roll number: %d\n", s.roll);
printf("Marks: %.1f\n", s.marks);

return 0;
}
```

[Run Code](#)

Output

```
Enter information:
Enter name: Jack
Enter roll number: 23
Enter marks: 34.5
Displaying Information:
Name: Jack
Roll number: 23
Marks: 34.5
```

Write a C program to using structures to Create an employee record with emp-no, name, age, date-of-joining (year), and salary. If there is 20% hike on salary per annum, compute the retirement year of each employee and the salary at that time. [standard age of retirement is 55]

C Program to Read and Print Employee Details

```
#include <stdio.h>
#include <stdlib.h>

typedef struct{
    char name[30];
    int id;
    double salary;
} Employee;
```

```

int main()
{
    //number of employees
    int n=2;

    //array to store structure values of all employees
    Employee employees[n];

    //Taking each employee detail as input
    printf("Enter %d Employee Details \n \n",n);
    for(int i=0; i<n; i++){
        printf("Employee %d:- \n",i+1);

        //Name
        printf("Name: ");
        scanf("%s",employees[i].name);

        //ID
        printf("Id: ");
        scanf("%d",&employees[i].id);

        //Salary
        printf("Salary: ");
        scanf("%lf",&employees[i].salary);

        //to consume extra '\n' input
        char ch = getchar();

        printf("\n");
    }

    //Displaying Employee details
    printf("----- All Employees Details ----- \n");
    for(int i=0; i<n; i++){
        printf("Name \t: ");
        printf("%s \n",employees[i].name);

        printf("Id \t: ");

```

```

    printf("%d \n",employees[i].id);

    printf("Salary \t: ");
    printf("%.2lf \n",employees[i].salary);

    printf("\n");
}

return 0;
}

```

The output of the Program:

```

"C:\Users\Win8.1\Saved Games\Employee\bin\Debug\Employee.exe"
Enter 2 Employee Details

Employee 1:-
Name: Rohan
Id: 1
Salary: 78453

Employee 2:-
Name: Sohan
Id: 2
Salary: 68743

----- All Employees Details -----
Name      : Rohan
Id        : 1
Salary    : 78453

Name      : Sohan
Id        : 2
Salary    : 68743

Process returned 0 (0x0)   execution time : 14.455 s
Press any key to continue.

```

Calculate Salary of the Employee from Basic

C program to read and print employee's record using structure/

```
#include <stdio.h>
```

```
/*structure declaration*/
```

```
struct employee{
    char name[30];
```

```
    int empId;
    float salary;
};

int main()
{
    /*declare structure variable*/
    struct employee emp;

    /*read employee details*/
    printf("\nEnter details :\n");
    printf("Name ?:");    gets(emp.name);
    printf("ID ?:");      scanf("%d",&emp.empId);
    printf("Salary ?:");  scanf("%f",&emp.salary);

    /*print employee details*/
    printf("\nEnter detail is:");
    printf("Name: %s" ,emp.name);
    printf("Id: %d" ,emp.empId);
    printf("Salary: %f\n",emp.salary);
    return 0;
}
```
